

Verified Generators and Relations for Qupit Clifford Operators

ANONYMOUS AUTHOR(S)

A recent paper by Bian, Li, Ross, van de Wetering, and Zhao gives a complete set of sixteen rewrite rules for multi-qudit Clifford circuits in every odd prime dimension p : two circuits over the gates H , S , and CZ implement the same unitary exactly when the rules prove them equal. The completeness proof is a normal-form argument in the tradition of Selinger’s presentation of the qubit Clifford group; its heart is a case analysis pushing every generator through every normal box, and its supporting derivations run to roughly a hundred pages. This paper reports a machine-checked proof of the result, in safe, axiom-free Agda, against an independently constructed semantics, together with the method that made it feasible. The formalisation does not follow the paper’s proof. The paper works with one rule set for the whole Clifford group and applies it, implicitly, “up to Paulis”; the step that promotes a symplectically complete rule set to a projectively complete one is convincing on paper but hard to turn into a verified normalisation procedure. We instead factor the projective Clifford group as a semidirect product $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$ of the projective Pauli group by the symplectic group, present the two factors separately, and glue the presentations with a generic, verified semidirect-product theorem. The symplectic factor carries all the mathematics: a unique normal form for $\text{Sp}(2n, \mathbb{Z}_p)$, refined into a doubly inductive datatype of *normal boxes* and *rows*, is realised as a tower of verified coset tables in the style of Reidemeister and Schreier, its uniqueness is proved by induction on the width against the symplectic action on Pauli vectors, and the paper’s 66 box relations are derived from eighteen natural ones. Presentation-level “plumbing” then reduces the glued alphabet and relations to the paper’s own generators and rules, and a verified isomorphism of presentations connects the two routes. Finally, the scalars are restored by a central extension, verified against a cocycle construction. Everything is parametric in the prime p and typechecks with no postulates; the semantic groups are constructed from $\mathbb{Z}_p$ arithmetic, and their identification with the groups of unitary matrices is the one step taken from the literature. The Agda development is about 100 000 lines, of which 60 000 are specific to the qupit Clifford groups.

CCS Concepts: • **Theory of computation** → **Type theory; Logic and verification; Equational logic and rewriting; Quantum computation theory.**

Additional Key Words and Phrases: Clifford group, qudits, generators and relations, completeness, normal forms, symplectic group, semidirect product, Agda, formal verification

ACM Reference Format:

Anonymous Author(s). 2026. Verified Generators and Relations for Qupit Clifford Operators. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 1 (October 2026), 27 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

A circuit calculus is a programming language of a particular kind: its programs are circuits, words of gates on numbered wires, and its equational theory is a finite list of circuit equations. Such a list is *sound* when each equation holds in the intended semantics and *complete* when every semantic equality between circuits is derivable from it. Completeness is what a language designer wants from an equational theory: it licenses normal-form-based decision procedures for program equivalence, and it tells the author of a circuit optimizer that no valid rewrite is missing. It is also, for quantum circuits, the kind of theorem that resists being trusted on paper. Soundness is a finite collection of matrix identities. Completeness is proved by a *normal form*: one shows that every circuit rewrites to a normal form using the equations, and that distinct normal forms denote distinct operators. The middle step is a case analysis, every generator against every piece of every normal form, that is mechanical, unforgiving and enormous. Selinger’s presentation of the n -qubit

2026. ACM 2475-1421/2026/10-ART1
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Clifford group [Selinger 2015] runs a substantial rewrite system through such an analysis; its qutrit successor [Li et al. 2025] needs more than fifty pages; and its generalisation to all odd primes by Bian, Li, Ross, van de Wetering, and Zhao [Bian et al. 2026a] — sixteen rewrite rules for multi-qudit Clifford circuits in every odd prime dimension p , henceforth “the qupit paper” — rests on roughly a hundred pages of derivations spread over its appendices and a supplement [Bian et al. 2026b].

This paper reports a machine-checked proof of that theorem, and, more importantly for a programming-languages audience, a *method* for machine-checking theorems of this shape. We formalise, in safe, axiom-free Agda [Norell 2007; The Agda Community 2026], that the qupit paper’s rule set presents the multi-qupit Clifford group at every width n and every odd prime p , against a semantics constructed independently of the syntax. The formalisation is built on a library for presentations, normal forms and completeness of circuit relation sets [Anonymous 2026a,b], whose design — inductively defined circuits, presented monoids as inductive congruences, normal forms as function bundles, a verified Reidemeister–Schreier engine, and presentation theorems for products and extensions — we recall on the library’s smallest worked example in §4. Every normal form in the development is an executable function, so each completeness theorem doubles as a verified decision procedure for circuit equivalence in its calculus.

The method. The formalisation does *not* follow the qupit paper’s proof, and the reason is the paper’s main methodological lesson. The paper works with a single rule set for the whole Clifford group and organises its completeness proof around the symplectic group $\text{Sp}(2n, \mathbb{Z}_p)$, the Clifford group modulo Paulis: it shows that every circuit rewrites, *up to Paulis*, to a symplectic normal form, and then boosts symplectic completeness to projective and unitary completeness by a general argument (its Proposition 4.9) in which each rewrite is repaired by pushing a Pauli correction to the end of the circuit. Throughout, relations are used “projectively”: Paulis and scalars are silently dropped and later restored. This is sound mathematics, and we believe it; but it is a poor specification for a verified normalisation procedure. The rewriting steps preserve no single syntactic invariant, the correction Paulis are not part of the data the rewrites compute with, and the termination and well-definedness arguments are stated at a level — “push the Pauli to the right, it changes to some other Pauli” — that a proof assistant would have to reconstruct from scratch.

We therefore chose a route that a proof assistant likes: *factor the group, present the factors, compose the presentations*. The projective Clifford group is a semidirect product of the projective Pauli group by the symplectic group, and the Clifford group is a central extension of that by the scalars. We present the three factors separately — the symplectic group by a coset-table normal form, the Pauli group compositionally, the scalars as a cyclic group — and glue the presentations together with generic, verified presentation theorems for semidirect products and central extensions. On this route the Paulis are never implicit: the symplectic development never mentions them, and they enter only through the conjugation action of the semidirect product, which is verified once, by semantics, rather than threaded through every rewrite. A final layer of presentation-level “plumbing” reduces the glued alphabet and relations to the paper’s generators H , S , CZ and its rules, and a verified isomorphism of presentations connects the two routes: the paper’s Figure 1 read modulo scalars presents the same group as our semidirect presentation.

1.1 Contributions

- (1) **A method, and a comparison.** We explain precisely where the qupit paper’s proof resists formalisation and how the factor-present-compose route avoids each difficulty (§3), walk through the library’s pipeline on its smallest case study, the swap circuits presenting the symmetric groups, and recall the product constructions it composes (§4), and report what

the formalisation covers and what it does not (§7). The route is generic: its two presentation theorems were proved for arbitrary groups before this project began.

- (2) **A machine-checked presentation of the projective qupit Clifford groups.** For every odd prime p and every width n , the paper's Figure 1 read modulo scalars presents $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$ (Theorem 2.1), via a chain of verified isomorphisms of presentations from a semidirect presentation on X, Z, H, S, CZ down to the paper's fifteen rules on H, S, CZ (§6).
- (3) **A machine-checked presentation of $\text{Sp}(2n, \mathbb{Z}_p)$ and a verified unique normal form.** Eighteen relations present the symplectic groups (Theorem 2.3); the normal form is the paper's, refined into a doubly inductive datatype of *normal boxes* and *rows*, realised as a Reidemeister–Schreier coset tower, and proved unique against the symplectic action on Pauli vectors (Theorem 2.2, §5).
- (4) **The relation reductions, verified.** The box relations that the normalisation uses are derived from the full symplectic rules, those from the simplified ones, the semidirect relations from the Clifford ones, and the paper's rules from ours; each reduction is a checked isomorphism of presentations (§5.5, §6.3).
- (5) **The scalars restored.** The exact rule set — the sixteen rules corrected by the phase each actually picks up, a single power of ω on one rule — presents a central extension of the projective group by $\langle \omega \rangle$, verified against the Weyl cocycle (§6.4).

Everything stated as a theorem in this paper typechecks under `-safe`: there are no postulates, no holes and no classical axioms. Two facts are *not* formalised, and we say so wherever they are used: that the projective Clifford group of unitaries is $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$, and that the Clifford group is a central extension of it by $\langle \omega \rangle$. Both are classical [Bian et al. 2026a], neither involves circuits, and §7 states exactly what our semantic groups are.

Outline. Section 2 recalls qupit Clifford operators and states the three main theorems as Agda types. Section 3 contrasts the paper's proof with ours. Section 4 walks through the worked example of permutation circuits and recalls the product constructions. Sections 5 and 6 are the method: the symplectic presentation, then the composition of the three factors. Section 7 gives statistics, engineering lessons and caveats; §8 surveys related work; §9 concludes.

2 QUPIT CLIFFORD CIRCUITS AND THE MAIN THEOREMS

2.1 From qubit to qudit Clifford gates

Fix an odd prime p and let $\omega = e^{2\pi i/p}$. A *qupit* is a p -dimensional quantum system with computational basis $|0\rangle, \dots, |p-1\rangle$, indices read in $\mathbb{Z}_p$. The single-qupit Pauli operators are $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$; the n -qupit Pauli group $\mathcal{P}_n$ is generated by tensor products of X and Z together with the phases ω^t . The n -qupit Clifford group C_n is generated, under composition and tensor product, by the gates of Table 1, whose qubit counterparts are shown for comparison. We follow the qupit paper's conventions [Bian et al. 2026a]: the $\frac{1}{2}$ in S is the inverse of 2 in $\mathbb{Z}_p$, and H carries a global phase δ_p chosen so that all three generators have determinant one (which is what makes the Clifford phases exactly the powers of ω).

Three quotients of C_n matter. Writing $\langle \omega \rangle$ for the scalars, $\overline{C}_n = C_n / \langle \omega \rangle$ is the *projective* Clifford group and $\overline{\mathcal{P}}_n = \mathcal{P}_n / \langle \omega \rangle \cong \mathbb{Z}_p^{2n}$ the *projective* Pauli group, an abelian group in which X and Z commute. Conjugation by a Clifford operator permutes Paulis and preserves their commutation phases, so modulo Paulis a Clifford operator is a symplectic matrix: $C_n / \mathcal{P}_n \cong \overline{C}_n / \overline{\mathcal{P}}_n \cong \text{Sp}(2n, \mathbb{Z}_p)$, the symplectic group of the $2n$ -dimensional $\mathbb{Z}_p$ -vector space $\mathbb{Z}_p^{2n}$ with its standard symplectic form [Bian et al. 2026a], and the projective Clifford group splits: $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$. These are

Table 1. The Clifford generators for qubits and for qupits (odd prime p).

	qubit ($p = 2$)	qupit (odd prime p)
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	$ j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk} k\rangle$
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	$ j\rangle \mapsto \omega^{j(j-1)/2} j\rangle$
CZ	$\text{diag}(1, 1, 1, -1)$	$ j\rangle l\rangle \mapsto \omega^{jl} j\rangle l\rangle$

the “glue facts” of the introduction; our formal development *defines* its semantic groups as the right-hand sides.

2.2 Main result 1: a presentation of the projective qupit Clifford group

In the formalisation a gate set is a family of gate types indexed by arity, and a circuit on n wires is a *word* over wire-indexed generators: $\text{Circuit } n = \text{Word } (\text{Gen } n)$ is the free monoid on the generators of H , S and CZ placed on each wire, $_ \uparrow$ shifts a circuit up one wire, and diagrams are read in matrix-multiplication order (§4 gives the generic machinery). The remaining usual gates are *words*, not extra generators: $Z = HSHHS^{-1}$, $X = HSHHS^{-1}H$, the swap $Ex = (CZ \cdot H \downarrow \cdot H \uparrow)^3$, and the controlled- X gates $CX = H \downarrow^3 \cdot CZ \cdot H \downarrow$ and XC with the wires exchanged, where $S^{-1} = S^{p-1}$, $H^{-1} = H^3$ and $CZ^{-1} = CZ^{p-1}$ are the words that the relations make inverses, and $_ \downarrow$ is the identity on circuits, there to make “the lower copy of a gate” visible in a rule. One derived gate has no qubit counterpart: the *multiplier* $M_x |j\rangle = |xj\rangle$ for a unit $x \in \mathbb{Z}_p^*$, which the qupit paper spells (its Lemma 2.14) as $HS^{g^{-1}}HS^gHS^{g^{-1}}$ followed by a Pauli correction and a global phase, and which we mostly spell $R^x H R^{x^{-1}} H R^x H$ with $R = S \cdot Z^{1/2}$, which absorbs the Pauli and drops the phase; the two spellings are interderivable (§6.3). Every Clifford module is parameterised by the prime, as $p = p-3 + 3$ with a primality proof (the symplectic modules take $p = p-2 + 2$, so Theorems 2.2 and 2.3 are proved for every prime), and by a primitive root g of $\mathbb{Z}_p^*$ with the witness g -gen that every unit is a power of g . So every theorem holds for every odd prime uniformly, and for the multiplicative generator chosen, exactly as the qupit paper fixes a generator g in its Figure 1; the existence of a primitive root modulo every prime is classical and is taken as a parameter rather than proved. ($\mathbb{Z}_p$ itself is $\text{Fin } p$ with a ring solver, Bézout inversion and Fermat’s little theorem; from g and g -gen the discrete logarithm and the exponent law $g^k = g^{k \bmod (p-1)}$ are derived.)

Our primary rule set — the qupit paper’s Figure 1 read modulo scalars, with the multiplier spelled over R and with $(SH)^3 = \varepsilon$ kept as an axiom — is, in Agda, one inductive family $_ \text{CRel}_{===}$ with sixteen constructors, one per rule, each stated at the smallest number of wires at which it makes sense; Figure 1 shows them as circuit equations, drawn from the Agda terms, under the constructor names used throughout. Six are one-qupit rules: the orders $S^p = \varepsilon$, $H^2 = M_{-1}$ and $(SH)^3 = \varepsilon$, the multiplier calculus M -power ($M_g^k = M_{g^k}$) and semi-MR ($R M_g = M_g R^{g^2}$), and comm-HSHHS. Seven are two-qupit rules: the orders of CZ and of the swap Ex , the commutations of CZ with S and M_g on the upper wire, the two rules moving S and H across the swap, and blake-c12, which relates two circuits implementing controlled- Z . Three are three-qupit rules: the Yang–Baxter and cz -slide rules of the swap calculus, and semi- $CX \uparrow \text{-} CZ \downarrow$. Beyond these, *every* circuit theory in the framework gets the same three structural rules for free: congruence under $_ \uparrow$, and gates on disjoint wires commute, one rule per arity (a global scalar gate, when there is one, gets a fourth: it is the same gate on every wire; that it then commutes with every wire gate is a theorem). Nothing else is an axiom: $XZ = ZX$

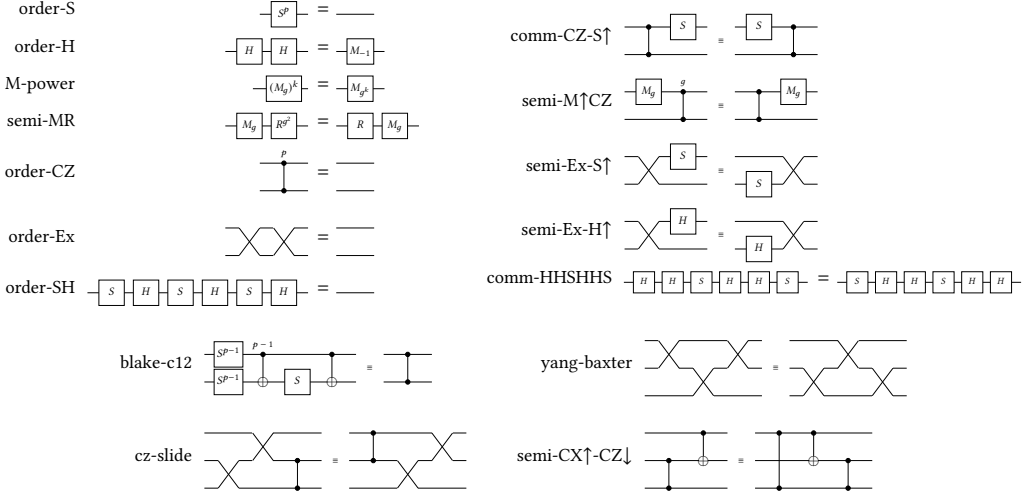


Fig. 1. The sixteen rules of `_CRel_===_` as circuit equations. A word is drawn left to right as written and read in matrix-multiplication order, so in $w \bullet v$ the right factor v acts first. `blake-c12` relates two circuits implementing controlled-Z; `yang-baxter` and `cz-slide` are the swap calculus.

modulo scalars, the commutation of `Ex` with `CZ`, and both rules for pushing a Pauli through `CZ` are *derived*. The count is sixteen where the paper’s Figure 1 has fifteen non-scalar rules because we keep $(SH)^3 = \epsilon$ as an axiom under the R -spelling; §6.3 shows that under the paper’s own S -spelling it is a theorem, and verifies the paper’s fifteen.

The first main theorem says that these rules *present* the projective Clifford group, at every width:

THEOREM 2.1 (CLIFFORD-PRESENTATION). *For every width n , the sixteen rules present the projective Clifford group:*

Theorem-1 : $\forall (n : \mathbb{N}) \rightarrow (n \text{ CRel_===_}) \text{ IsPresentationOf } (\text{Pauli} \rtimes \text{Sp } n)$

Theorem-1 = `PapPres.presentation`

Home: `ProjectiveClifford.Qupit.Paper-V0.Presentation and Paper-V1.Presentation`
(*module paths here are under Examples.Groups*).

Here $\text{Pauli} \rtimes \text{Sp } n$ is the semidirect product of the projective Pauli group $\mathbb{Z}_p^{2n}$ by the symplectic group, constructed in §6.2, and `_IsPresentationOf_` is the record of §4.4: a proof that every generator has an inverse, an interpretation `[[_]]` of words in the group, and a *group isomorphism* from the words modulo the rules onto the group. Its two halves are the two directions of the diagram

$$\text{Cir } n / \approx_{16 \text{ rules}} \xrightarrow{[[_]] \cong} \overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p) \cong C_n / \langle \omega \rangle :$$

soundness — `[[_]]` is well defined, each rule holds semantically — and *completeness* — `[[_]]` is injective, semantically equal circuits are related by the rules. The same theorem is proved for the paper’s fifteen rules in their own spelling (§6.3) and, with the scalar ω and its one correction added, for the exact Clifford group (§6.4).

2.3 Main result 2: a unique normal form for symplectic operators

Symplectic operators are Clifford operators modulo Paulis — the “proper” part of a Clifford operator, its stabiliser tableau. Our normal forms for them are *data*, built from four kinds of atomic boxes:

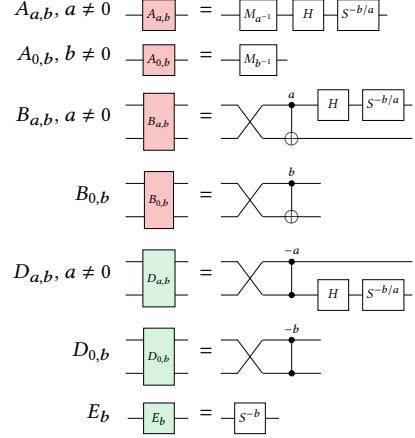
A B D E : Set

$A = \Sigma[ab \in (\mathbb{Z}_p \times \mathbb{Z}_p)] (ab \neq (0, 0))$

$B = \mathbb{Z}_p \times \mathbb{Z}_p$

$D = \mathbb{Z}_p \times \mathbb{Z}_p$

$E = \mathbb{Z}_p$



The boxes are the paper's normal boxes (its Figure 6), with the same names; the pictures show how each is spelled as a circuit. Rows of boxes and the staircase of rows are two more inductive definitions (§5.2), and a normal form is read back as a circuit by structural recursion. The second main theorem says the normal form is unique for the symplectic semantics of §5:

THEOREM 2.2 (UNIQUE-NF). *Normal forms with equal symplectic denotations are equal:*

Theorem-2 : $\forall n \{u \ v : NF\ n\} \rightarrow \llbracket [u] \rrbracket \approx^s \llbracket [v] \rrbracket \rightarrow u \equiv v$

Theorem-2 = $U.\llbracket [] \rrbracket$ -injective

Home: Symplectic.Normalization.Uniqueness.

Soundness plus a unique normal form yields completeness, by one generic lemma (§4.4); and the hard, interesting part of the whole development is to find these coset representatives, parameterised by the number of wires, and to push every generator through them (§5).

2.4 Main result 3: a verified presentation of the symplectic operators

The symplectic group $\text{Sp}(2n, \mathbb{Z}_p)$ itself is modelled as the group Sp -group n of symplectic transformations of Pauli vectors (§5.1), and the third main theorem says that a small rule set over H , S and CZ alone presents it: the *simplified* symplectic rule set of §5.5, fifteen group-specific rules — six rules for the order of S and the multiplier calculus of the single multiplier M_g of the primitive root ($H^2 = M_{-1}$, $M_g^k = M_{g^k}$, and M_g past S and, on either wire, past CZ), then order- CZ , comm- CZ - S ↓, comm- CZ - S ↑ and Selinger's c10–c15 [Selinger 2015] — plus the three structural rules, eighteen in all. It is obtained from the *full* symplectic rule set of seventeen rules — Selinger's c10–c15 verbatim, order- CZ and the two CZ - S commutations, and the eight-rule multiplier calculus over arbitrary units, the set to which §5.5 reduces the box relations — by an identity isomorphism of presentations: the two sets share an alphabet and generate the same congruence, so the theorem proved for the full rules transfers to the simplified ones with the same interpretation.

THEOREM 2.3 (SIMPLIFIED-PRESENTATION). *For every width n , the simplified symplectic rules present the symplectic group:*

Theorem-3 : $\forall \{n\} \rightarrow (n \text{ SRel_} ____) \text{ IsPresentationOf } (\text{Sp-group } n)$

Theorem-3 = $\text{SimPres.presentation}$

Home: Symplectic.Simplified.Presentation.

The statement is verbatim from its home module, where `_SRel,===_` is the full relation set: in `Simplified.Syntactics` the fifteen group-specific rules form a family of that name too — the one §5.5 displays — and `_QRel,===_` is its closure under the three structural rules by *Lift-Relation*, over which `MainTheorems` states the theorem; the home module re-exports the lifted relation under the shorter name, exactly as `_CRel,===_` in Theorem 2.1 is the lifted relation of `Paper-V0`. The proof has the shape of the diagram above, one level down: soundness is one clause per axiom; completeness is Theorem 2.2 through the generic *by-normalization* — the rules rewrite every circuit to its normal form, and semantically equal circuits have equal normal forms; and surjectivity, that every symplectic transformation is the action of some circuit, is “clear the bottom wire, recurse” (§5.6). Theorem 2.1 is then built on this theorem: the semidirect product of §6.2 assembles it with a presentation of the projective Pauli group and the conjugation action into the presentation of $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$.

3 TWO ROUTES TO COMPLETENESS

The two routes share their mathematics, the same normal boxes, the same symplectic normal form, and essentially the same eighteen symplectic relations, and differ in the *organisation* of the completeness argument, which is what a proof assistant cares about.

3.1 The paper’s route

The qupit paper [Bian et al. 2026a] proves that the sixteen rules of its Figure 1 are sound and complete for the unitary interpretation in three moves. *A normal form*: its Section 3 defines a normal form for an n -qupit Clifford circuit as a *symplectic normal form* followed by a *Pauli normal form*; the former, built from the normal boxes A, B, D, E in alternating Z -normal and X -normal layers, represents the stabiliser tableau, the latter is the Pauli correction. Proposition 3.8 shows that symplectic normal forms are in bijection with $\text{Sp}(2n, \mathbb{Z}_p)$, with proofs “analogous to those for the qutrit case”. *Symplectic completeness*: its Section 4.1 rewrites any circuit to symplectic normal form by pushing the gates in one by one; pushing a gate through a normal box yields an updated box and residual *dirty* gates, which are pushed further in. Collecting every case gives 66 parametrised *box relations*; termination in a clean normal form is Lemma 4.2, proved by inspecting which dirty gates each box can absorb; and Theorem 4.4 reduces the 66 box relations to 18 symplectic relations, a step the paper already delegated to Agda. Throughout, relations are applied *up to Paulis*: the symplectic interpretation identifies a circuit with the same circuit followed by any Pauli. *Boosting*: its Proposition 4.9 lifts symplectic completeness to projective completeness: given a symplectically complete rule set and a *Pauli complete* one (which derives the Pauli identities and the rules for pushing a Pauli through a generator), correct each symplectic rule $C_l = C_r$ to $C_l = C_r ; P$ for the unique Pauli P that makes it projectively sound, and the union is projectively complete. The proof replays a sequence of symplectic rewrites with the corrected rules, after each step pushing the newly created Pauli to the end of the circuit, where it turns into “a potentially different Pauli”. The same argument with $\langle \omega \rangle$ in place of $\mathcal{P}_n$ gives unitary completeness.

3.2 Why we did not formalise that

Each move is correct, and each was stated at exactly the level of precision a careful reader needs. A proof assistant needs more, and the gap is widest at the places where the paper says “analogous”, “by inspection” and “potentially different”.

First, the rewriting procedure is not a function. “Push the gates in one by one” describes a non-deterministic rewrite relation whose termination is argued by an invariant on the placement of dirty gates, verified by reading the 66 box relations. To formalise Proposition 4.3 one would define

the normalisation as a structurally recursive function on circuits, prove it sound with respect to the relations, and prove that its result is a clean normal form — and at that point one has written a coset table, which is what we do (§5.3), except that the coset table would also have to say what happens to the Pauli.

Second, and more seriously, the projective use of relations means that the objects the rewriting manipulates are not circuits but circuits modulo Paulis, without any syntactic representation of the quotient. A box relation $g M = M' \text{ dir}$ holds *up to a Pauli* that the relation does not name. Formalising this faithfully means either carrying the Pauli explicitly through every box relation, turning each of the 66 relations into one with a computed correction and re-doing the reductions $66 \rightarrow 18 \rightarrow 16$ with corrections, or working in a syntax where Paulis are genuinely trivial. The paper does the former informally, in Proposition 4.9, by observing that the corrections *can* be computed and pushed to the end. The honest formal counterpart of that observation is a normalisation procedure on the semidirect product whose well-definedness — that congruent inputs give equal cosets and congruent residuals — is exactly what “potentially different Pauli” glosses over: the Pauli produced depends on the rewrite path, and one must prove that all paths agree. We know this works; we did not see how to implement it and verify it at reasonable cost.

Third, the uniqueness proof, Proposition 3.8, is by reference to the qutrit paper; formalised, it is a statement about the coset representatives at each level of a tower, and it is where the doubly inductive structure of our normal form pays off (§5.4).

3.3 Our route: factor, present, compose

Two short exact sequences organise the qubit Clifford group:

$$\begin{array}{ll} 1 \rightarrow \overline{\mathcal{P}}_n \rightarrow \overline{C}_n \rightarrow \text{Sp}(2n, \mathbb{Z}_p) \rightarrow 1 & \text{semidirect: it splits;} \\ 1 \rightarrow \langle \omega \rangle \rightarrow C_n \rightarrow \overline{C}_n \rightarrow 1 & \text{central extension.} \end{array}$$

We work in a syntax where Paulis are genuinely trivial — the symplectic circuits of §5 have *no* Pauli generators, only H , S , CZ , and their semantics is $\text{Sp}(2n, \mathbb{Z}_p)$ itself — and reintroduce Paulis and scalars afterwards by construction. Figure 2 shows the chain. Reading upward: the symplectic group is presented by a coset tower whose cosets are the paper’s normal boxes arranged as a doubly inductive type, whose table is the paper’s “push a gate through a box” as a structurally recursive function with a verified section, and whose well-definedness is proved *semantically*, by uniqueness of the normal form one level down (§5); the projective Pauli group is presented by three rules and glued on by the semidirect theorem, whose only new obligation, that the conjugation action respects both factors’ rules, is discharged by soundness and completeness of the two factor presentations rather than by hand (§6.2); plumbing reduces the semidirect alphabet and relations to the paper’s, and two identity isomorphisms carry the result to our sixteen rules and to the paper’s fifteen (§6.3); and the scalars are restored by the extension theorem, with the one correction computed and verified against a cocycle (§6.4).

Does this just move the difficulty around? No. The symplectic factor is where the real rewriting happens, and keeping the Pauli part out of it pays off everywhere: every box relation is an honest equation, every coset-table entry is a function, and the Pauli bookkeeping that the paper’s Proposition 4.9 performs by hand is performed once, by the semidirect-product theorem, in a form that was proved for arbitrary groups before this project began. The factored route is, we think, the right way to organise such proofs even on paper.

4 A WORKED EXAMPLE: PERMUTATION CIRCUITS

We begin with the smallest complete case study in the library: circuits built from a single two-wire gate, the swap σ , which present the symmetric groups. The walkthrough exhibits the pipeline

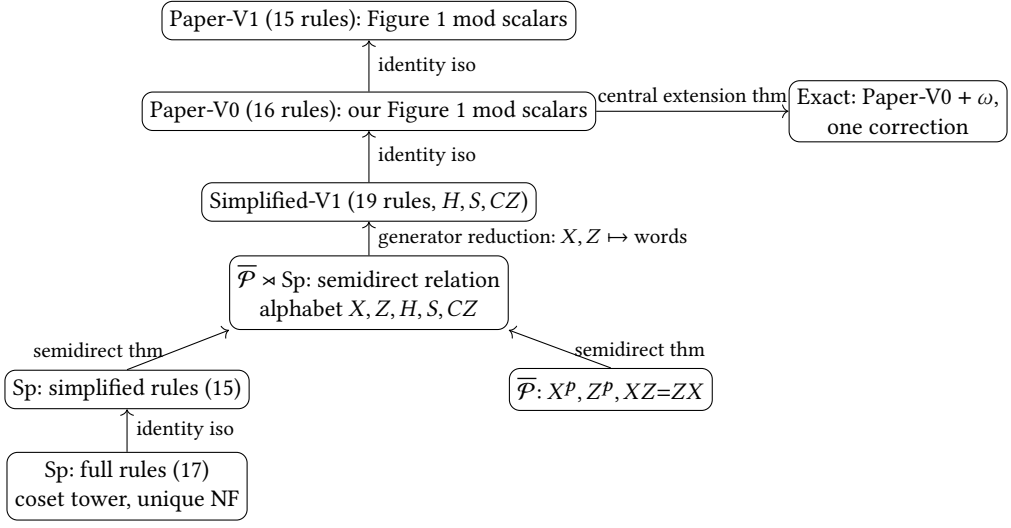


Fig. 2. The chain of presentations. Every arrow is a verified presentation theorem or a verified isomorphism of presentations; the semantic group is $\text{Sp}(2n, \mathbb{Z}_p)$ at the bottom, $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$ from the semidirect level up, and a central extension of it by $\langle \omega \rangle$ at the top right.

— gates, relations, cosets, tower, semantics, uniqueness, completeness, presentation — that §5 instantiates again for the symplectic factor, concretely enough that the latter can be read as the same pipeline with richer gates and a richer action. The section closes with the product constructions (§4.5) by which §6 glues such presentations together.

4.1 Gates, generators, and circuits

A gate set is a family $\text{Gate} : \mathbb{N} \rightarrow \text{Set}$ giving, for each arity, the gates on that many adjacent wires; for permutations there is one gate, on two wires. The generic circuit layer of the library turns any such family into wire-indexed *generators*: a gate placed at the bottom of the wires, or a generator shifted up by one wire.

```

data Gate : ℕ → Set where
  σ-gate : Gate 2
data Gen : ℕ → Set where
  gate1 : Gate 1 → Gen (1+ n) — a 1-ary gate on the bottom wire
  gate2 : Gate 2 → Gen (2+ n) — a 2-ary gate on the bottom two wires
  ↑ : Gen n → Gen (1+ n) — the same generator, one wire higher

```

Here $1_+ n$ and $2_+ n$ are the library's patterns for $1 + n$ and $2 + n$, and n is an implicit argument, so gate_2 σ -gate is a generator at any width from two upward. An n -wire *circuit* is a word of generators, $\text{Circuit } n = \text{Word } (\text{Gen } n)$: a single generator $[g]^w$, the empty circuit ε — n bare wires, the identity circuit at that width — or a sequential composition $c \bullet c'$. We write σ for the one-letter circuit $[\text{gate}_2 \text{ } \sigma\text{-gate}]^w$ and $c \uparrow$ for c shifted up one wire ($\uparrow = \text{wmap } \uparrow$). Read diagrammatically, with wires drawn bottom-up and a word drawn left to right as written (as in Figure 1; the permutation semantics of §4.4 applies the leftmost gate first), σ is a crossing of the bottom two wires, $\sigma \uparrow$ the same crossing one wire higher, and $\sigma \bullet \sigma \uparrow \bullet \sigma$ the three-wire braid on the left of the yang-baxter equation drawn beside its axiom below.

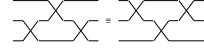
4.2 Relations: two axioms, structure for free

The group-specific relations are an inductive family indexed by the wire count. Each axiom is stated once, with its gates at the bottom of the wires and the width left open (the implicit n), so that one constructor covers every width at which the axiom fits; the structural rule $\text{cong}\uparrow$ below moves it to any height. (Agda names may be mixfix: each underscore in $_S\text{Rel}, _===$ marks an argument position, so the family is applied as $n \text{ SRel}, w === v$, read “ $w = v$ is an axiom on n wires”.)

```
data _SRel, _===_ : (n : ℕ) → WRel (Gen n) where
```

```
  order : (2+ n) SRel, σ • σ === ε
```

```
  yang-baxter : (3+ n) SRel, σ • σ ↑ • σ === σ ↑ • σ • σ ↑
```



These two axioms are two thirds of the Coxeter presentation of S_n : $\sigma^2 = \varepsilon$ and the braid (Yang–Baxter) relation. The third Coxeter relation, that swaps on disjoint wires commute, is not specific to permutations: it is one of the relations every circuit theory needs, whatever its gates. By a *circuit theory* we mean a gate set with gate-specific relations, presenting for each width n a monoid of n -wire circuits under sequential composition. Such circuits are the fixed-width slices of a symmetric monoidal category (a PROP), and the monoidal structure leaves exactly two traces at fixed width: the shift $c \mapsto c \uparrow$ is “tensoring with an identity wire”, so equations must be preserved by it, and the interchange law says that gates on disjoint wires commute. (The library also admits 0-ary gates — global scalars, available at every width — with one more structural rule, that a scalar is identified with its own shift, $\omega \uparrow = \omega$; that it then commutes with every generator is a theorem, once scalars are assumed to commute with one another, which is vacuous for a single scalar.) These are the structural rules every circuit theory shares — what a pen-and-paper presentation leaves implicit in “a congruence compatible with composition and tensor product” — and the circuit layer adds them generically: Lift-Relation extends any gate-specific family to the full relation set $_V\text{Rel}, _===$:

```
data _VRel, _===_ : (n : ℕ) → CRel n where
```

```
  srel : n SRel, w === v → n VRel, w === v
```

```
  cong↑ : n VRel, w === v → (1+ n) VRel, w ↑ === v ↑
```

```
  comm1 : (h : Gate 1) (g : Gen n) → (1+ n) VRel, [ g ↑ ]w • [ gate1 h ]w === [ gate1 h ]w • [ g ↑ ]w
```

```
  comm2 : (h : Gate 2) (g : Gen n) → (2+ n) VRel, [ g ↑↑ ]w • [ gate2 h ]w === [ gate2 h ]w • [ g ↑↑ ]w
```

From here on, $\approx$ denotes the monoid congruence generated by $n \text{ VRel}, _===$: the least relation containing the axioms and closed under reflexivity, symmetry, transitivity, congruence of $\bullet$, associativity, and the unit laws; deciding it is the word problem. Because σ is its own inverse and shifts of invertible generators are invertible, the presented monoid is a group; we witness this by a Grouplike proof (every generator has a left inverse), by induction on generators.

4.3 Cosets and the staircase normal form

The engine behind the normal form is coset enumeration along the chain $S_1 < S_2 < \dots < S_n < S_{n+1}$, where S_n sits inside S_{n+1} as the permutations fixing the bottom wire — syntactically, the circuits of the form $c \uparrow$. Read as permutations, two circuits lie in the same right coset of S_n exactly when they bring the same wire to the bottom position; there are $n+1$ choices, so S_n has index $n+1$, and a natural representative of each coset is the *descending staircase* of swaps that carries that wire down. In the library a coset is not a set of circuits but a label naming one, an element of the finite type $C \ n$; here the label is the length of the staircase, as a unary numeral bounded by the width, and the *section* $[_]^\text{c}$ maps it to the staircase circuit. The representatives on three, two, and one wires are drawn on the right:

data $C : \mathbb{N} \rightarrow \text{Set}$ where

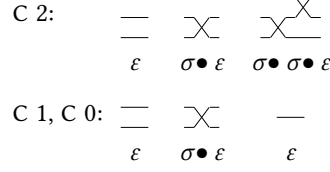
$\varepsilon : C\ 0$

$\sigma \bullet _ : C\ n \rightarrow C\ (1 + n)$

$[_]^c : C\ n \rightarrow \text{Circuit}\ (1 + n)$

$[\varepsilon]^c = \varepsilon$

$[\sigma \bullet c]^c = \sigma \bullet ([c]^c \uparrow)$



So $C\ n$ has exactly the $n+1$ elements drawn above for $n = 2, 1, 0$, matching the index of S_n in S_{n+1} . Why this yields a normal form is the classical argument of computational group theory [Holt et al. 2005; Sims 1994]: every $f \in S_{n+1}$ lies in one coset, say with representative c_n , so $f = f' \cdot c_n$ with $f' \in S_n$, and recursing on f' writes f uniquely as $c_1 c_2 \cdots c_n$ with $c_k \in C\ k$ – a numeral in the factorial number system, one digit per level, since $|S_{n+1}| = (n+1) \cdot n \cdots 2$.

Computing this factorisation is the job of the coset table. Given a coset, represented by its staircase, and one generator, the table says where their product lands – the updated coset – and what is left over on the wires above – the residual circuit – so that staircase followed by generator equals residual (shifted up) followed by updated staircase. Pushing σ into a staircase can lengthen it, cancel with it ($\sigma^2 = \varepsilon$), or pass it through and shift it up (Yang–Baxter), or commute past; for a generator that does not touch the bottom wire we can recurse. The residual is empty or a single swap here, but in richer gate sets it can be a longer circuit, which is why the table returns a circuit on one wire fewer rather than a generator (Figure 3 draws the four cases):

$\text{ract} : C\ (1 + n) \rightarrow \text{Gen}\ (2 + n) \rightarrow \text{Circuit}\ (1 + n) \times C\ (1 + n)$

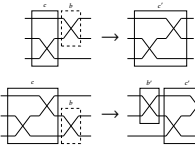
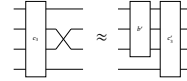
$\text{ract}\ \varepsilon\ \sigma\text{-gen} = \varepsilon, \sigma \bullet \varepsilon$ – grows

$\text{ract}\ (\sigma \bullet \varepsilon)\ \sigma\text{-gen} = \varepsilon, \varepsilon$ – cancels ($\sigma^2 = \varepsilon$)

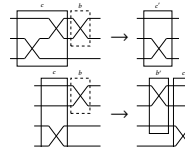
$\text{ract}\ (\sigma \bullet \sigma \bullet c)\ \sigma\text{-gen} = \sigma, \sigma \bullet \sigma \bullet c$ – passes (Yang–Baxter)

$\text{ract}\ \varepsilon\ (g \uparrow) = [g]^w, \varepsilon$ – commutes past

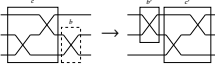
$\text{ract}\ (\sigma \bullet c)\ (g \uparrow) = \text{proj}_1 (\text{ract}\ c\ g) \uparrow, \sigma \bullet (\text{proj}_2 (\text{ract}\ c\ g))$ – recurses



grows



cancels ($\sigma^2 = \varepsilon$)



passes (Yang–Baxter)



commutes past

Fig. 3. Pushing a swap through a staircase: the four cases of the coset table ract as circuit rewrites.

$\sigma\text{-gen}$ abbreviates $\text{gate}_2\ \sigma\text{-gate}$. The last clause is the recursion: a shifted generator meets a staircase by meeting its tail one wire down. Each clause is certified by the *soundness law*, pictured on the right and proved once per table by a short equational derivation from the axioms:

$\text{ract-sound} : \forall\ c\ b \rightarrow \text{let}\ (b', c') = \text{ract}\ c\ b\ \text{in}\ [c]^c \bullet [b]^w \approx b' \uparrow \bullet [c']^c$

The stateful word traversal $_t$ extends ract to whole circuits, threading the coset from left to right and concatenating the residuals; run from the identity coset, it maps an $(n+2)$ -wire circuit to an $(n+1)$ -wire circuit and a coset – one level of normalisation.

What the library checks a table against is fixed once and for all. One level is a Reidemeister–Schreier situation: a subgroup presentation Γ and a group presentation Δ , a finite type C of coset labels with an identity coset I , a section $[_] : C \rightarrow \text{Word}\ Y$ mapping each label to its representative, the generator embedding f , and the coset table $h : C \rightarrow Y \rightarrow \text{Word}\ X \times C$, which pushes one letter of the group past a coset representative and returns the residual word of the subgroup

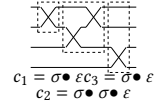
and the updated coset. The classical algorithms *find* such a table; the library takes a claimed table and *verifies* it against five hypotheses: (1) the table inverts the embedding at the identity coset; (2) it respects Δ 's axioms, coset by coset (h-wd-ax); (3) the embedding f respects Γ 's axioms; (4) the identity coset is represented by ε , $[I] \approx \varepsilon$; and (5) the *soundness law*, that sliding one letter past a representative matches the table. From these it derives the normal-form transport $\text{NormalForm } \Gamma \text{ NF} \rightarrow \text{NormalForm } \Delta \text{ (NF} \times \text{C)}$, and *CosetTower* iterates it up an $\mathbb{N}$ -indexed family of presentations, one level per wire. Here Γ and Δ are the relations on $n+1$ and $n+2$ wires, f is the shift, h is ract, and (5) is ract-sound; (3) is the structural rule $\text{cong}\uparrow$ itself, (1) and (4) hold by evaluation, and (2) is a case analysis over the axiom instances, coset by coset, whose coset halves hold by evaluation and whose residual halves are one-line derivations (the unit laws, or the axiom being pushed). For the symplectic group even the coset half of (2) does not close by evaluation, and §5.3 explains what replaces it.

Iterating the level is the coset tower, whose carrier grows by one digit per level:

$\text{NF} : \mathbb{N} \rightarrow \text{Set}$

$\text{NF } 0 = \mathbb{T}$

$\text{NF (suc } k) = \text{NF } k \times \text{C } k - \mathbb{T} \times \text{C } 0 \times \text{C } 1 \times \dots \times \text{C } k$



so $\text{NF } n$ has exactly $n!$ elements. The normal-form function $\text{nf} : \text{Circuit } n \rightarrow \text{NF } n$ runs the tables; its *section* $\text{inv-nf} : \text{NF } n \rightarrow \text{Circuit } n$ rebuilds a circuit from its digits by concatenating the staircases, suitably shifted, as drawn on the right for $(\text{tt}, c_0, c_1, c_2, c_3) \in \text{NF } 4$, the digit $c_0 \in \text{C } 0$ being the empty staircase — one staircase per level, each shifted up to the wires of its level: a “staircase of staircases”.

4.4 Two semantics, uniqueness, and completeness

So far everything was syntax. The development interprets circuits twice: by a *loose* semantics $[[_]] : \text{Circuit } n \rightarrow (\text{Fin } n \rightarrow \text{Fin } n)$ into endofunctions of $\text{Fin } n$ (σ swaps the bottom two elements), a monoid that is *not* a group; and by a *tight* semantics into $\text{Permutation}'\text{-group } n$, the standard library's permutations of $\text{Fin } n$ bundled as a *Group* by our *ForStdlib* layer. *Soundness* — $w \approx v$ implies $[[w]] \doteq [[v]]$ — is a small case analysis over the axioms. The heart of the matter is the *unique normal form* property, that normal forms with equal denotations are equal: $[[\text{inv-nf } u]] \doteq [[\text{inv-nf } v]]$ implies $u \equiv v$, proved by induction on the tower: the lower digits fix the last point, so the top digit alone determines where the permutation sends it; equal denotations therefore force equal top digits, and cancelling them reduces the claim to the level below. Soundness and uniqueness are exactly the premises of the generic lemma by-normalization, which concludes *completeness*: $[[w]] \doteq [[v]]$ implies $w \approx v$. Unfolding the lemma on this example shows where the axioms enter. Each step of nf is one of the rewrites of Figure 3, and each of those is an instance of order, of yang-baxter, or of the structural rules (the far commutation comm_2 , lifted to the wires above by $\text{cong}\uparrow$); so $w \approx \text{inv-nf } (\text{nf } w)$ — the law of the *NormalForm* witness — is a derivation from exactly those rules. Now suppose $[[w]] \doteq [[v]]$. By soundness the normal forms of w and v have the same denotation, so by uniqueness they are equal, and hence $w \approx \text{inv-nf } (\text{nf } w) = \text{inv-nf } (\text{nf } v) \approx v$, a derivation assembled from the two normal-form derivations and nothing else. Completeness thus never appeals to a rule beyond the axioms and structural rules above. Examples. Groups. Symmetric states these for every n . Completeness for the *loose* semantics is the stronger, perhaps surprising fact: even read as mere endofunctions, circuits that are equal as functions are provably equal. For the tight semantics, soundness, completeness, and surjectivity (every permutation is a product of adjacent transpositions) assemble into the library's *presentation record*. The record packages two things: a *Grouplike* witness, which makes the presented monoid a group, and a proof that the semantics $[[_]]$ is a group isomorphism from that group onto the target. We quote this one statement in Agda because every presentation theorem in the paper has its shape:

`symmetric-presentation` : $\forall n \rightarrow (n \text{ WRel_} ==_) \text{ IsPresentationOf (Permutation'-group } n)$

read: the monoid of swap circuits modulo $\{\text{order, yang-baxter}\}$ and the structural rules *is* the group of permutations of $\text{Fin } n$ (`IsPresentationOf` is the mixfix record name `_IsPresentationOf_` used infix). The symplectic factor of §5 is this pipeline again, with richer gates and a richer action, and §6 composes such presentations.

4.5 Products of presentations, and their presentation theorems

The presentation just obtained is one factor of the groups §6 assembles; this last subsection recalls the constructions that glue such factors together. Relation sets over a disjoint union of alphabets (`Presentation.Construct.Base`) are built from a single primitive, a three-way join whose *mixed* component is the design parameter:

```
data _◇_ (Γ₁ : WRel A) (Γ₂ : WRel B) (Γ₃ : WRel (A ⊔ B)) : WRel (A ⊔ B) where
  left : Γ₁ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ] [ v ]ₗ
  right : Γ₂ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]ᵣ [ v ]ᵣ
  mid : Γ₃ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) u v
```

Choosing the mixed relation recovers the classical constructions — free product (`EmptyRel`), direct product (`CommRel`: generators of the two sides commute), semidirect product (`ConjRel`/`ConjRelW`: moving h past n conjugates n , by a generator or by a word), and definitional extension (`SugarRel`: a new generator desugars to a word) — plus n -fold iterations $\Gamma \oplus^\wedge n$. Congruence-lifting lemmas (lefts, rights) embed the factors' congruences into the join, and normal-form transports carry witnesses across these maps: a normal form for one factor becomes a weak normal form for the union, and a normal form for a presented monoid pulls back along any monoid monomorphism into it.

For each construction, `Presentation.Construct.Properties` proves two kinds of theorems. *Normal-form lifting*: given normal-form witnesses for the factors Γ and Δ , with carriers NF_1 and NF_2 , the construction has a normal-form witness whose carrier is $NF_1 \times NF_2$ for the direct and semidirect products. *Presentation*: if the factors present concrete groups, the constructed relation set presents the constructed group. With their premise telescopes elided, the direct, semidirect and n -fold statements read:

```
dpres : (Γ ◇ Δ ◇ CommRel) IsPresentationOf (G₁ × G₂)
dpres : (Γ ◇ Δ ◇ ConjRelW conj) IsPresentationOf (G₁ ⋈ G₂)
presentation : ∀ n → (Γ ⊕∧ n) IsPresentationOf (⊗-group n)
```

where the semantic groups on the right are standard-library-style bundles built by the library's `ForStdlib` layer. The semidirect theorem is the one §6 uses, so we state it in full. Its mixed relation `ConjRelW conj` says that moving a letter h of the acting group past a letter n of the normal subgroup conjugates n , by a *word*: $h \bullet n = \text{conj } h \ n \bullet h$. The theorem takes the two relation sets and the action, two well-definedness conditions on the action — it respects the acting group's axioms in its first argument (`conj-hyph`) and the normal subgroup's in its second (`conj-hypn`) — and presentations of the factors, and returns a presentation of the semidirect product of the two *semantic* groups:

```
module _ (conj-hyph : ∀ {c d} n → c ==₂ d → (conjh) c n ≈₁ (conjh) d n)
  (conj-hypn : ∀ c {w v} → w ==₁ v → (conjn) c w ≈₁ (conjn) c v) where
  module Presentation (G1 G2 : Group 0f 0f)
    (p1 : Γ IsPresentationOf G1) (p2 : Δ IsPresentationOf G2) where
    dpres : (Γ ◇ Δ ◇ ConjRelW conj) IsPresentationOf G1⋈G2
```

Here $G1 \times G2$ is built by the library's standard-library style semidirect product from an Action record whose action is the syntactic `conj` transported through the two presentation isomorphisms; its

normal form is the product of the factors' normal forms, and uniqueness reduces factor by factor to the injectivity of the two factor interpretations (unfp_3). A further module proves the classical recipe for presenting a group *extension*: if $1 \rightarrow N \rightarrow G \rightarrow Q \rightarrow 1$ is a short exact sequence, and presentations of N and Q are given together with the data that realises it in G — how G conjugates N , and the correction in N by which each lifted relator of Q fails to hold — then N 's relations, word-valued conjugation rules, and the corrected lifts of Q 's relators together present G (cf. Proposition 2.55 of [Holt et al. 2005]). A second mixed component, RelTwist , twists each relator $u = v$ of the quotient by its correction word in the normal subgroup, and the relation set of the recipe — the one the qupit paper invokes, in its pen-and-paper form, for its Section 4.2 — is

$$\text{extension-presentation } S R \text{ conj corr} = S \diamond \text{EmptyRel} \diamond (\text{ConjRel}^w \text{ conj} \cup \text{RelTwist } R \text{ corr})$$

and the library proves it as a presentation theorem for any group extension realised by a short exact sequence and, in the form used in §6.4, for central extensions built from a cocycle. Semidirect products are the special case with trivial corrections. These construction theorems are what make the composition step of §6 cheap: the semidirect product of the Pauli and symplectic presentations and the central extension by the scalars are *assembled* from the factor presentations, with no fresh coset enumeration at all.

5 THE SYMPLECTIC FACTOR

This section presents the symplectic factor, where the real rewriting happens, in six steps: the semantics; the normal form; the coset table that rewrites to it, which is the S_n table of §4.3 under a richer action; uniqueness; the reduction of the relations the table uses to the eighteen rules that present $\text{Sp}(2n, \mathbb{Z}_p)$; and the presentation theorem they assemble into (§5.6).

5.1 The semantics: symplectic transformations of Pauli vectors

Rather than $2n \times 2n$ matrices, the formalisation models $\text{Sp}(2n, \mathbb{Z}_p)$ as the group of $\mathbb{Z}_p$ -linear, symplectic-form-preserving bijections of the phase space $\text{Pauli } n = \text{Vec } (\mathbb{Z}_p \times \mathbb{Z}_p) \ n$ — one pair (a, b) , the exponents of $Z^a X^b$ (up to phase), per wire. A symplectic transformation is a record $\text{Symplectic } n$ carrying the action ap , its inverse, and proofs that the two are inverse, that ap is linear, and that it preserves the symplectic form sform . Equality is extensional equality of the action alone, so the proof fields never obstruct. Composition is function composition, and therefore associativity and the unit laws hold *on the nose*, by refl — a small choice with large consequences downstream, since every monoid case of every soundness proof closes definitionally. $\text{Sp-group } n$ is the resulting Group. The generators act as the qupit paper's conjugation tables (its Lemmas 2.22–2.23) read on exponent vectors, and $[[_]]$ is the homomorphic extension:

```
actg : ∀ {n} → Gen n → Pauli n → Pauli n
actg (gate1 H-gate) ((a, b) :: ps) = (- b, a) :: ps
actg (gate1 S-gate) ((a, b) :: ps) = (a, b + a) :: ps
actg (gate2 CZ-gate) ((a, b) :: (a', b') :: ps) = (a, b + a') :: (a', b' + a) :: ps
actg (g ↑) (x :: ps) = x :: actg g ps
```

Words are read left to right, so $w \bullet v$ applies w first and its denotation composes on the right. The seven proof obligations per generator — inverses, linearity, preservation of the form — are identities in $\mathbb{Z}_p$, discharged with the ring solver. Soundness of every axiom of §5.5 in this semantics is one clause per axiom in SoundnessDirect (1 100 lines).

5.2 The normal form: boxes, rows, and the staircase

The qupit paper's symplectic normal form is a Z -normal layer followed by an X -normal layer on n wires, then a normal form on $n - 1$ wires. We refine this into a doubly inductive datatype. The atomic boxes were shown in §2; rows and the staircase are:

$ML : \mathbb{N} \rightarrow \text{Set}$

$M 0 = \top$

$M (1_+ n) = \text{Vec } D \ n \times E$

$L 0 = \top$

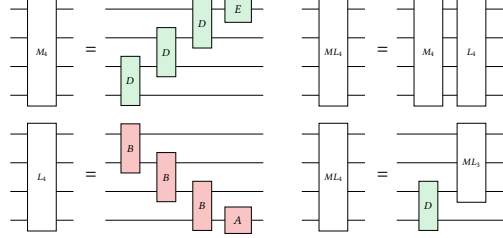
$L (1_+ n) = \text{Vec } B \ n \times A$

$ML 0 = \top$

$ML 1 = M 1 \times L 1$

$ML m @ (2_+ n) = M m \times L m$

$\uplus D \times ML (1_+ n)$



An M -row is a column of D boxes ending in an E box (the paper's X -normal layer); an L -row is a column of B boxes ending in an A box (its Z -normal layer, with the A box at the bottom). An ML box at width $m \geq 2$ is *either* a full pair of rows *or* a bottom D box followed by an ML box one wire up: a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

$NF : \mathbb{N} \rightarrow \text{Set}$

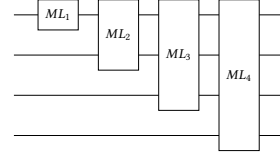
$NF 0 = \top$

$NF (1_+ n) = NF \ n \times ML (1_+ n)$

$[\] : \forall \{n\} \rightarrow NF \ n \rightarrow \text{Word } (\text{Gen } n)$

$[\] \{0\} \quad \text{tt} \quad = \varepsilon$

$[\] \{1_+ n\} (nf, ml) = [nf] \uparrow \bullet [ml]^{ml}$



Reading normal forms back as circuits is structural recursion, box by box. The box-to-word maps make the pictures of §2 precise; for instance

$[\]^d : \forall \{n\} \rightarrow D \rightarrow \text{Word } (\text{Gen } (2_+ n))$

$[\]^d \{n\} (0, b) = \text{Ex} \bullet \text{CZ}^\wedge (-b)$

$[\]^d \{n\} (a @ (1_+ _), b) = \text{Ex} \bullet \text{CZ}^\wedge (-a) \bullet H \bullet S^\wedge -b/a$

where $-b/a = -b \cdot a^{-1}$, and an A box $A_{a,b}$ with $a \neq 0$ is $\text{XM } a \bullet H \bullet S^{-b/a}$, a multiplier, a Hadamard, and an S -power. Why the spine? The paper's Z -normal circuit $W_Z^{(n)}$ has its A box on some wire i , with B boxes above it and nothing below; the position i is an index that the paper's normal-form circuits carry implicitly. The ML type makes it explicit as the length of the D -spine, and, crucially, makes the coset type at width m a *sum* of the coset type at width $m - 1$ and a flat case — exactly the shape the coset table and the uniqueness proof recurse on. The identity coset is the flat one with all exponents zero and $A_{0,1}$ at the bottom, and the section sends it to ε up to $\approx$, because $\text{Ex}^2 \approx \varepsilon$ collapses each $D_{0,0}$ and $B_{0,0}$.

5.3 From symmetric to symplectic: change the action

The coset table of §4.3 serves the symplectic group under a richer action. There are three ways of looking at the wires. S_n acts on $\{0, \dots, n - 1\}$: a wire is a position, and $c \circ f^{-1}$ fixes 0. S_n acts on $\text{Vec Bit } n$: a wire is a bit, and $c \circ f^{-1}$ fixes the $\mathbb{Z}_2$ -linear subspace of the bottom wire. Adding H, S, CZ , the group acts on $\text{Vec Pauli}_1 \ n$, a $2n$ -dimensional symplectic $\mathbb{Z}_p$ -vector space in which a wire is a two-dimensional symplectic subspace; enlarging the coset layer from staircases to ML boxes, for each $f \in \text{Sp}(2n, \mathbb{Z}_p)$ there is a c such that $c \circ f^{-1}$ fixes the symplectic subspace of the bottom wire. The

cosets at width $1+n$ are the ML boxes, their section is $[_]^{ml}$, and the coset table is the paper's "push a gate through a normal box", now a function $\text{ract} : ML(1+n) \rightarrow \text{Gen}(1+n) \rightarrow \text{Circuit } n \times ML(1+n)$ with the same soundness law $[c]^c \bullet g \approx b' \uparrow \bullet [c']^c$ as in §4.3. ract dispatches on the shape of the coset — a flat box or a D -spine — and on the generator — S or H on the bottom wire, CZ on the bottom two, or a lifted gate — and delegates each case to a module of the Pushing directory (26 files, 5 600 lines): S and H through a flat box, CZ through a flat box, CZ meeting a D box and a flat box, a lifted gate through the spine. The residual b' lives one wire up, as the type says. The soundness proof ract-sound is a chain of $\approx$ -steps per clause, each step a box relation or a structural rule. For example, an S on the bottom wire meeting a spine $D_{a,b}$ followed by lm : S commutes past the lifted lm , then the box relation $D \leftarrow S$ rewrites $D_{a,b} \bullet S$ to $\text{dir} \uparrow \bullet D_{a,b-a}$, and re-association finishes.

So far this is the S_n story with bigger cosets. The difference is in the five hypotheses of §4.3. Four close as before; the second, h-wd-ax — that the table respects the axioms, i.e. congruent inputs produce equal cosets and congruent residuals — does *not* close by evaluation, because the axioms of $\text{Sp}(2n, \mathbb{Z}_p)$ (unlike $\sigma^2 = e$) are not decided by a finite computation on the coset data. The formalisation proves it in two halves. The *coset half* is proved semantically: apply $[[_]]$ to ract-sound on both sides of an axiom, observe that residuals live one wire up and so cannot move the bottom wire, and conclude from head-injectivity of the coset section (§5.4) that the two cosets are equal. The *residual half* then follows by cancellation — provided the shift $\uparrow$ is injective on congruence classes at the lower width, which is a consequence of faithfulness (completeness) there. This makes the tower a well-founded induction on the width, $\text{tower} : \forall n \rightarrow \text{TowerLevel } n$, where a level packages the normal form at that width together with the agreement of its section with the coset sections: a normal form at width k gives faithfulness at k (by by-normalization); faithfulness gives injectivity of $\uparrow$ on congruence classes (using that a lifted word acts as $\text{id} \oplus [[w]]$); that discharges h-wd-ax at width k ; and the generic normal-form transport produces the normal form at width $k+1$. The base case, width 1, is built by hand. This is the formal counterpart of the paper's Lemma 4.2 and Proposition 4.3: no termination argument is needed beyond structural recursion, and "the dirty gates eventually get removed" is the type of ract .

5.4 Uniqueness of the normal form

Theorem 2.2 says that normal forms with equal denotations are equal. The action of a normal form on a Pauli vector peels the staircase one wire at a time — the coset factor acts, the resulting bottom entry is fixed, and the inner normal form acts on the tail — and injectivity is a structural induction on the staircase:

```
lemma-lm-head-inj : ∀ {n} (lm1 lm2 : ML (1+ n)) →
  (∀ (ps : Pauli (1+ n)) → head (act [ lm1 ]ml ps) ≡ head (act [ lm2 ]ml ps)) → lm1 ≡ lm2
lemma-lm-tail-surj : ∀ {n} (lm : ML (1+ n)) (qs : Pauli n) →
  ∃ λ (ps : Pauli (1+ n)) → tail (act [ lm ]ml ps) ≡ qs
```

Two obligations. *Tail surjectivity* is free, because a symplectic transformation carries its inverse. *Head injectivity* — the bottom-wire output determines the coset — is the crux, and it is itself an induction on the ML type (LMHeadInj , 1 300 lines): at width 1 a case analysis on the four shapes of an A box using closed forms for the actions of S^k , HS^k and the multipliers; a separation lemma showing a flat box and a spine never agree on heads; and, for two spines, the D box is read off by probing with the two basis Paulis Z and X on the bottom wire and the identity above, after which the hypothesis descends one width. With these two lemmas the generic $\text{Circuit.Uniqueness}$ module of the library assembles $[[[]]]$ -injective and the UniqueNormalForm witness (the unique-normal-form property of §4.4); the module's header records the one subtlety, that ap is a *left* action — ap of a

composite applies its right factor first — so the coset factor of a normal form is the one next to the input, which is what lets the coset be recovered from every state rather than from states of a restricted shape. This is the paper’s Proposition 3.8 with its Lemmas 3.4–3.7, now an induction whose structure is dictated by the datatype.

5.5 Relation reduction: box relations and the two rule sets

Which relations does *ract-sound* use? Rewriting *every* circuit to normal form and collecting every relation the rewrite ever uses gives the paper’s box relations — one per gate per box shape, parametrised by the box’s labels. The formalisation organises them as nine families of case tables, each a lemma quantified over the box parameters with a computed direction *dir-of* and updated box:

wires	family	pushed
1	$A \leftarrow H, A \leftarrow S, E \leftarrow S$	H or S through an A box; S through an E box
2	$B \leftarrow H\uparrow, S\uparrow, S; D \leftarrow H, S\uparrow, S, CZ; L \leftarrow CZ$	a gate through a B , a D , or an L -row
3	$BB \leftarrow CZ\uparrow, B\uparrow \leftarrow CZ, DD \leftarrow CZ$	CZ through two B or two D boxes

For instance, the two-wire table for a D box states that $[d]^d \bullet g \approx S^e \bullet \text{dir}\uparrow \bullet [d']^d$ with e , dir and d' computed from the labels of d and the gate g — the paper’s Figure 18, with the residual split into an S -power on the bottom wire and a word one wire up. The nine families are the paper’s Figures 15–21; unfolding the case tables gives 42 parametric relations, against the paper’s count of 66, the difference being only in how cases are grouped (the formalisation, for instance, proves each of the $\uparrow/\downarrow$ pairs of a B box and a D box once and obtains the other by a duality lemma). The width-3 relations are stated at width 3 and widened to arbitrary width by a congruence $\text{cong}\downarrow^k$.

The box relations are derived (BR, 7 000 lines, on top of a calculus of *Ex*-conjugations and two-qubit lemmas of 11 000 lines) from the *full* symplectic rule set, seventeen group-specific axioms whose two- and three-qubit members are Selinger’s c10–c15 [Selinger 2015] verbatim and whose one-qubit members are the multiplier calculus — $S^p = H^4 = (SH)^3 = \varepsilon$, $H^2S = SH^2$, and four schemes over units x, y : $M_x M_y = M_{xy}$, $M_x S = S^{x^2} M_x$, and M_x on either wire past CZ gives CZ^x — with M_x spelled $S^x H S^{x^{-1}} H S^x H$. By inspection one finds a smaller set of primitive, bidirectional relations that derive all of these: the *simplified* symplectic rule set, which replaces the schemes by statements about the single multiplier M_g of the primitive root, strengthens $H^4 = \varepsilon$ to $H^2 = M_{-1}$, and drops $(SH)^3 = \varepsilon$ and $H^2S = SH^2$:

```

order-S : (1+ n) SRel, S ^ p == ε
order-H : (1+ n) SRel, H ^ 2 == M_1
M-power : ∀ k → (1+ n) SRel, Mg ^ k == M (g ^ k)
semi-MS : (1+ n) SRel, Mg • S == S ^ (g * g) • Mg
semi-M↑CZ : (2+ n) SRel, Mg ↑ • CZ == CZ ^ g • Mg ↑
semi-M↓CZ : (2+ n) SRel, Mg ↓ • CZ == CZ ^ g • Mg ↓

```

plus the same order-CZ, comm-CZ-S↓, comm-CZ-S↑ and selinger-c10 to c15: fifteen group-specific rules, or eighteen with the three structural ones. The choice is not canonical; one picks the set one finds natural. The two rule sets generate the same congruence, and since they share an alphabet the identity on words is an isomorphism of presentations in both directions (*Simplified.Iso*); the substantive direction derives each full axiom from the simplified ones. The multiplier law $M_x \bullet M_y \approx M_{xy}$ is typical: write $x = g^k$, $y = g^l$, use *M-power* twice, add the exponents, reduce modulo $p - 1$ using $M_g^{p-1} \approx \varepsilon$, which is Fermat’s little theorem, and use *M-power* once more. The whole reduction from seventeen rules to fifteen is 900 lines; the reduction from the box relations to the seventeen is 18 000. This is the formal counterpart of the paper’s Theorem 4.4, whose

proof the paper already delegated to Agda. Read modulo Paulis and scalars, Selinger’s three-qubit relations [Selinger 2015] are *identical* to selinger-c12 to c15, which is why the names were kept, and his two-qubit relations are similar; the one-qubit relations are where the odd prime shows, in the multiplier calculus of M_g with its dependence on a primitive root and on Fermat’s theorem, which has no qubit counterpart. That the difficulty of the general odd prime concentrates on one wire is what makes the factored route affordable: the two- and three-wire rewriting is Selinger’s, and the one-wire arithmetic is arithmetic.

5.6 The symplectic presentation theorem

Soundness (one clause per axiom), the normal form of §5.3 and its uniqueness (§5.4) assemble, through by-normalization, into a sub-presentation: the interpretation is an injective homomorphism from the words modulo the full rules into Sp-group. Surjectivity — every symplectic transformation is the action of some circuit — is proved constructively by “clear the bottom wire, recurse on the tail”: the single-level input is

Theorem-LM : $\forall \{n\} (p\ q : \text{Pauli } n) \rightarrow \text{sform } p\ q \equiv 1 \rightarrow$
 $\exists \lambda (lm : \text{ML } n) \rightarrow \text{act } [lm]^{ml} p \equiv pZ_0 \times \text{act } [lm]^{ml} q \equiv pX_0$

— for any symplectic pair there is a coset box sending it to the bottom-wire Z and X — which is the paper’s Lemma 3.7 with the box computed rather than asserted. Composing the sub-presentation of the full rules with the identity isomorphism of §5.5 gives Theorem 2.3 (§2.4), the presentation theorem for the simplified rules, with the *same* interpretation.

6 COMPOSING: PAULIS, THE SEMIDIRECT PRODUCT, AND SCALARS

From the symplectic presentation to the Clifford presentation there are four steps: the Pauli factor; the semidirect product, gluing the Pauli and symplectic presentations along the conjugation action; plumbing, which simplifies the glued presentation to the paper’s rules over H, S, CZ alone; and the central extension by $\langle \omega \rangle$, which restores the scalars.

6.1 The Pauli factor

The projective Pauli group $\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ has an easy presentation over generators X, Z on each wire:

data `_PRel,===_` : $(n : \mathbb{N}) \rightarrow \text{WRel } (\text{Gen } n) \text{ where}$
`order-X` : $(1 + n) \text{ PRel, } X \wedge p \equiv \varepsilon$
`order-Z` : $(1 + n) \text{ PRel, } Z \wedge p \equiv \varepsilon$
`comm-Z-X` : $(1 + n) \text{ PRel, } Z \bullet X \equiv X \bullet Z$

Theorem-Pauli : $\forall \{n\} \rightarrow (\text{XZ_QRel,===_ } n) \text{ IsPresentationOf } (+_p\text{-group } n)$

Theorem-Pauli = `XZ.presentation`

The semantic group $+_p\text{-group } n$ is $\text{Vec } (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ under componentwise addition — the very phase space on which $\text{Sp}(2n, \mathbb{Z}_p)$ acts in §5.1, which is what will make the semidirect action definable. The normal form is $X^a Z^b$ on each wire, read off the exponent vector; soundness, uniqueness and by-normalization give the presentation, and the module also exports the soundness and completeness maps separately, which the next step consumes. (The library also presents the same group compositionally, as an n -fold direct product of $\mathbb{Z}_p \times \mathbb{Z}_p$ with no normal-form work at all; that presentation is over a different alphabet, and the circuit-alphabet one is the one the qubit chain needs.)

6.2 The semidirect product

This is the semidirect construction of §4.5, $\Gamma \diamond \Delta \diamond \text{ConjRel}^w \text{ conj}$, instantiated with the Pauli presentation of §6.1 as the normal factor and the symplectic presentation of Theorem 2.3 as the acting one: the relations of both factors plus twisted commutation $h n h^{-1} = \text{conj} h n$. Here the action of a symplectic generator on a Pauli generator is the paper’s Lemmas 2.22–2.23, as words — H swaps X and Z (up to an inverse), S sends X to XZ , CZ couples the two bottom wires, and a shifted gate acts on a shifted Pauli only:

```

conj : Sym.Gen n → XZ.Gen n → Word (XZ.Gen n)
conj Sym.H-gen X-gen = Z
conj Sym.H-gen Z-gen = X ^ p-1
conj Sym.S-gen X-gen = X • Z
conj Sym.S-gen Z-gen = Z
conj Sym.CZ-gen X-gen = X • Z ↑
conj Sym.CZ-gen Z-gen = Z
conj Sym.CZ-gen (X-gen ↑) = X ↑ • Z
conj Sym.CZ-gen (Z-gen ↑) = Z ↑
conj Sym.H-gen (xz ↑) = [ xz ]w ↑
conj Sym.S-gen (xz ↑) = [ xz ]w ↑
conj Sym.CZ-gen (xz ↑ ↑) = [ xz ]w ↑ ↑
conj (sym ↑) X-gen = X
conj (sym ↑) Z-gen = Z
conj (sym ↑) (xz ↑) = conj sym xz ↑

Pauli×Symplectic : (n : ℕ) → WRel (XZ.Gen n ⊔ Sym.Gen n)
Pauli×Symplectic n = (n PRel,===) ◊ (n SpRel,===) ◊ ConjRelw conj

```

(module qualifiers on the Pauli side elided; the relation is `_QRel,===_` in the source).

The generic theorem of §4.5 asks for two things beyond the factor presentations: that `conj` respects the symplectic rules in its first argument and the Pauli rules in its second. Neither is proved by hand — by a case split over the axioms and the generators, as one would for a simpler action such as S_n permuting the factors of $\mathbb{Z}_N^n$ in a wreath product. Both go through the semantics: the action of a symplectic word c on a Pauli word w , computed syntactically by (conj^h) , denotes $\text{ap} [[c]] (\text{sem } w)$ in the phase space, so if $c \approx d$ by a symplectic axiom then the two conjugates have equal denotations by symplectic soundness, and are therefore congruent Pauli words by Pauli *completeness*:

```

respects-Δ : ∀ {n} {c d : Word (Gen n)} (u : Word (XZ.Gen n)) → Sim._QRel,===_ n c d →
  PB._≈_ (XZ._QRel,===_ n) ((conjh) c u) ((conjh) d u)
respects-Δ {n} {c} {d} u ax =
  complete n (Eq.trans (conjw-sem c u) (Eq.trans (denote-eq ax (sem u)) (Eq.sym (conjw-sem d u))))

```

and symmetrically for `respects-Γ`. The long symplectic rules — M -power, $\text{semi-}M\uparrow CZ$, selinger-c10 to c15 — are never conjugated by hand. This is the first place the factored route pays off in a way the paper’s route cannot: the paper’s “Pauli complete” rules (its Definition 4.7) are precisely these conjugation rules, and there they must be derived syntactically from Figure 1 (its Proposition 4.8). With every premise a theorem, the presentation $(\text{SemiDirect} _QRel,===_ n) \text{ IsPresentationOf Pauli} \times \text{Sp}$ is unconditional, where $\text{Pauli} \times \text{Sp}$ is the theorem’s $G1 \times G2$.

The semantic side. $\text{Pauli} \times \text{Sp}$ is the library’s semidirect product of the two semantic groups, built from an Action record (§4.5) whose action is the syntactic `conj` transported through the two presentation isomorphisms. The *natural* semidirect product, with $\text{Sp}(2n, \mathbb{Z}_p)$ acting on $\mathbb{Z}_p^{2n}$ by ap , is a different term, and the two are identified by a lemma `act≡ap` proving that the transported action is ap . This lemma lives in an abstract block: with it transparent, the file’s header records, a two-line

lemma about Z^j “went from 27 s to out-of-memory at 12 GB”, because Agda tried to unfold the transported action through both presentation isomorphisms.

6.3 Plumbing: presentation simplification

Two presentations are *equivalent* when the presented groups are isomorphic — when each side’s axioms are derivable from the other’s. The semidirect presentation has five generators per wire, X, Z, H, S, CZ , and the relations of Table 2; the paper has three, H, S, CZ , and sixteen relations. Getting from one to the other is *generator reduction* — X and Z become the derived words of §2 — and *relation reduction* — the Pauli rules and the fourteen conjugation rules become derivable. For example, $CZ \bullet X\uparrow \approx X\uparrow \bullet Z\downarrow \bullet CZ$ (conj CZ-gen (X-gen $\uparrow$)) becomes a theorem of the Clifford rules.

Table 2. The three presentations of the projective Clifford group and the two reductions between them.

	SemiDirect	Simplified-V1	Paper-V0 / Paper-V1
generators	X, Z, H, S, CZ	H, S, CZ	H, S, CZ
relations	Pauli (3) + symplectic (15) + conjugation (14)	19	16 / 15
<i>generator reduction</i> : X, Z become the derived words; $S \mapsto R = S \cdot Z^{1/2}$			
<i>relation reduction</i> : Pauli + conjugation rules become derivable; c10–c15 and the swap calculus interderivable			

From the semidirect relation to Simplified-V1. The first reduction is an isomorphism of presentations on *different* alphabets, given by generator maps in both directions: f sends X and Z to their derived words, H and CZ to themselves, and S to R ; h sends H and CZ to themselves and S to $Z^{-1/2} \bullet S$; both commute with $\uparrow$. The symplectic S is carried to $R = S \cdot Z^{1/2}$ and h compensates, because the target rule set, Simplified-V1, spells the multiplier over R , as §2 does: $M_x = R^x H R^{x-1} H R^x H$ exactly, with no Pauli correction. Simplified-V1 has nineteen group-specific rules: the simplified symplectic rules of §5.5 with R in place of S in the multiplier rules and in c10, c11, plus $(SH)^3 = \varepsilon$, $H^2 S H^2 S = S H^2 S H^2$, and the two rules for pushing X through CZ . Notably $XZ = ZX$ is *not* among them; it is derived from $(SH)^3 = \varepsilon$ and M-power, by the paper’s own argument (its Proposition 4.8). The isomorphism theorem needs the four generator-level facts of the library’s StarGroupsIsomorphism: f and h respect the axioms (f-well-defined, h-well-defined, one clause per axiom, the former containing the derivations of all three Pauli rules and all fourteen conjugation rules), and each is a left inverse of the other on generators. Composing with the semidirect theorem:

presentation : $\forall \{n\} \rightarrow$
 (Clifford-Relations_QRel_==_n) IsPresentationOf (Semidirect.Pauli $\times$ Sp n)

This is where the mathematics of the composition step lives (about 6 000 lines): a multiplier calculus, a Pauli-conjugation calculus, the CZ layer, and a small rewriting tactic that discharges the bookkeeping the paper’s hand derivations spend most of their length on.

From Simplified-V1 to our sixteen rules. Paper-V0 is the rule set of §2: the same alphabet, so the identity on words is the candidate isomorphism. Ten of the axioms are shared verbatim; the rest is real work in both directions. Paper-V0 axiomatises the swap directly and two-qubit interactions through blake-c12 and semi-CX $\uparrow$ -CZ $\downarrow$, and from these its Lemmas (4 750 lines) derives Selinger’s c10–c15 natively together with the lower-wire rules, the two Pauli-versus- CZ rules and the swap- CZ commutation; conversely the seven swap rules are proved in Simplified-V1 by transport along the chain symplectic $\rightarrow$ simplified $\rightarrow$ semidirect $\rightarrow$ V1 rather than by fresh derivations. The module exports $v1 \Rightarrow pap$, which transports any Simplified-V1 theorem into Paper-V0’s theory; it is what lets later layers reuse the calculi rather than re-prove them.

From our sixteen rules to the paper's fifteen: connecting the two routes. The paper's Figure 1 spells the multiplier over S , with an explicit Pauli prefix (its Lemma 2.14): $M_x = \text{SHS}' x^{-1} x$ where $\text{SHS}' a b = Z^{(b-1)/2} X^{(1-a)/2} S^b H S^a H S^b H$, and correspondingly states semi-MR as $M_g \bullet S^{g^2} \bullet Z^{(g^2-g)/2} = S \bullet M_g$ (its rule C4), the Z -exponent being what is left after the $Z^{1/2}$ hidden in each R is split off and carried across the multiplier, where it picks up a factor of g . Paper-V1 is Figure 1 modulo scalars in this spelling: our sixteen rules with order-H, M-power, semi-MR and semi-M $\uparrow$ CZ restated over SHS' , and fifteen rules rather than sixteen, because $(SH)^3 = \varepsilon$ is now a theorem. Under the S -spelling, M_1 is the word $(SH)^3$ — at $x = 1$ the Pauli prefix vanishes — and M-power at $k = 0$ already gives $M_1 \approx \varepsilon$; under our R -spelling M_1 is $(RH)^3$, a different word, which is why the axiom had to stay there. The isomorphism with Paper-V0 is again the identity on words; eleven axioms cross directly, and the four multiplier rules need the bridge between the two spellings, $R^a H R^b H R^a H \approx \text{SHS}' a b$, proved once over an abstract relation in Shared.PauliBase and transported by $v1 \Rightarrow \text{pap}$. The result is the theorem the index module states:

clifford-presentation :

$$\forall n \rightarrow (\text{PapV1.Clifford-Relations._QRel._} == _ n) \text{ isPresentationOf } (\text{Pauli} \times \text{Sp } n)$$

So the paper's Figure 1, read modulo scalars, presents the same group as the semidirect presentation of §6.2: the two routes — the paper's single rule set with its projective use of relations, and our factored construction — provably arrive at the same place.

6.4 Getting the scalars back

The Clifford group is a central extension of the projective Clifford group by the scalars $\langle \omega \rangle$. Following the extension recipe of §4.5, the projective presentation is modified in four moves: add a generator ω ; add the relation $\omega^p = \varepsilon$; add relations that ω commutes with every gate; and *correct* each projective relation by the scalar it actually picks up. In Agda the recipe is instantiated rather than spelled out: extension-presentation is applied to the cyclic relation $\langle \omega \mid \omega^p = \varepsilon \rangle$, the projective rules, the constant conjugation $\text{conj } _ = \omega$, and a correction function that is ε on every rule except order-SH, where it is $\omega^{(p^2-1)/8}$. Reading the generators as the paper's matrices, with H normalised by δ_p so that all determinants are one, exactly *one* of the sixteen rules fails to hold on the nose: $(SH)^3 = \varepsilon$ picks up $\omega^{(p^2-1)/8}$ (cf. the phase in the paper's (T1)), which is exact division since $8 \mid p^2 - 1$ for odd p , and whose residue modulo p is $-1/8$ — so the corrected rule reads

$$\boxed{S} \boxed{H} \boxed{S} \boxed{H} \boxed{S} \boxed{H} \equiv \omega^{-1/8}$$

where $1/8$ is the inverse of 8 in $\mathbb{Z}_p$. Every other rule holds exactly: the three multiplier rules and order-H because $R = S \cdot Z^{1/2}$ has no Pauli part, blake-c12 because every Pauli in it is pure-Z, and pure-Z Paulis have vanishing symplectic form. Note that the quotient here is our sixteen-rule set; correcting the paper's fifteen would give Figure 1 itself.

The semantic side. A central extension of a group Q by an abelian group K is determined by a 2-cocycle $c : Q \times Q \rightarrow K$, with multiplication $(k, g) \cdot (k', h) = (k + k' + c(g, h), gh)$. The formalisation builds two. The *structural* one is the Weyl cocycle $c((P, S), (Q, _)) = \frac{1}{2} \text{sform}(P, \text{ap } S \ Q)$ on the natural semidirect product, whose cocycle law is symplectic invariance of sform , and which satisfies the Heisenberg law $P \cdot Q = \omega^{\text{sform}(P, Q)} Q \cdot P$ on Paulis — what distinguishes it from the direct product $\langle \omega \rangle \times (\mathcal{P}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p))$. The *presented* one, Presented-group, has as quotient the word group of the sixteen rules and as kernel the word group of $\langle \omega \mid \omega^p \rangle$, and its cocycle is generated from *generator data*: for each gate a and word v , the scalar by which the lifts of a and v fail to compose, together with the proof that this descends to the quotient and assigns each relator the same correction on both sides. The generator data is obtained by pulling the Weyl cocycle back

along the projective presentation theorem, so the two cocycles agree by construction. The theorem, again on no hypothesis, is

```
presentation-exact : ∀ (n : ℕ) →
  let Clifford-group = λ n → PE.Presented-group n (exact-generator-data n)
  in (n Exact,===) IsPresentationOf (Clifford-group n)
```

Its proof needs no normal forms — since both factors are word groups, $[[_]]$ is a splitting rather than a normalisation — but it does need, for each of the sixteen rules, that the phase the Weyl cocycle assigns to the left-hand side equals the correction plus the phase of the right-hand side. This reduces to nineteen equations in $\mathbb{Z}_p$ about an explicit phase function on circuits, discharged by Pauli arithmetic, by a one-wire calculus that reads a circuit as a triple (Pauli, 2×2 symplectic matrix, phase) with an explicit concatenation law, and, for order-SH, by the number-theoretic fact that the residue of $(p^2 - 1)/8$ is $-1/8$. The extension is provably non-split. This is the formal counterpart of the paper’s remark before its Theorem 4.10 that a complete rule set for $\langle \omega \rangle$ “only needs $\omega^d = 1$ ”, with the one non-trivial correction computed and checked.

7 DISCUSSION

7.1 What is verified, and what is not

Everything stated as a theorem in §§2–6 is checked by Agda 2.8 with the standard library 2.4, under `-safe`, with no postulates and no holes; the projective chain is `-cubical-compatible` as well. The headline theorems — `unique-nf`, `simplified-presentation`, `v1-presentation`, `clifford-presentation` — are restated with full types in the index module `MainTheorems.agda`, which reaches, through them, everything those proofs rest on; the exact presentation `presentation-exact` is checked from its own root. Every theorem is parametric in the odd prime p , the width n , and a primitive root g .

Three things are *not* formalised, and they delimit what “a machine-checked proof of the qubit paper’s theorem” means here. *The glue facts.* Our semantic groups are constructed: $\text{Sp}(2n, \mathbb{Z}_p)$ is the group of linear symplectic bijections of $\mathbb{Z}_p^{2n}$, $\overline{\mathcal{P}}_n$ is $\mathbb{Z}_p^{2n}$ under addition, $\text{Pauli} \rtimes \text{Sp}$ is their semidirect product, and the Clifford group with scalars is a central extension of that by a cocycle. That these are isomorphic to the groups of unitary matrices generated by the paper’s H , S , CZ modulo (or not modulo) phases is classical — the paper’s Proposition 2.25 and Theorem 2.26, citing Weil — and is not formalised. No complex number, matrix or unitary group appears in the qubit development; a verified matrix semantics would close this gap (§9). *The identification of the two central extensions.* The structural Clifford group (Weyl cocycle on the natural semidirect product) and the presented one have quotients that are provably isomorphic and cocycles that agree by construction; that the two *total* groups are isomorphic is not stated as a theorem. *The primitive root.* Every module takes a generator g of $\mathbb{Z}_p^*$ and the witness `g-gen` as parameters, mirroring the paper, whose Figure 1 is stated for a chosen g ; the existence of a primitive root modulo every prime is not in the library.

7.2 Statistics

Table 3 gives the shape of the development. The reusable framework is about 15 000 lines; the qubit-specific development is about 60 000, of which the symplectic factor alone is 35 000 — confirming that the symplectic group is where the work is — and the composition step 23 000. For comparison, the paper’s own appendices and supplement are roughly a hundred pages. Checking the exact presentation, with the interfaces of the projective chain cached, takes about six minutes on a laptop.

7.3 Proof-engineering lessons

Prove well-definedness by semantics. The single most consequential decision in the symplectic development is that the coset table’s respect for the axioms — the paper’s “the dirty gates get

Table 3. Size of the development (lines of Agda incl. comments, wc -l, September 2026).

Component	Files	Lines
Framework (Word, Presentation, Circuit, Normalization, ForStdlib)	54	14 848
Symplectic group (normalisation 8 352; box relations 18 161; soundness etc. 8 886)	118	35 399
Projective Pauli group	3	1 220
Projective Clifford: semidirect, plumbing, shared calculus	26	17 996
Clifford with scalars	11	5 029
Other examples in the tree (symmetric, cyclic, wreath, Clifford+T, qubit Clifford)	107	27 552
Not on the chain (Simplified-V2), index, notations	10	4 344
Total	329	106 388

removed in the same way whichever rewrite one applies” — is proved not by analysing the table but by uniqueness one level down: two congruent inputs produce cosets with the same bottom-wire action, so by head injectivity the same coset; the residuals then agree by cancellation, given injectivity of $\uparrow$, which is faithfulness one wire below. This turns the coset tower into a well-founded induction on the width and removes any need to reason about rewrite paths. *Transport, do not re-prove.* Every rule set above the semidirect level is connected to the one below by an isomorphism of presentations, and theorems are carried across by congruence lemmas rather than re-derived; calculi shared by several rule sets are stated once over an abstract relation with the differing facts as hypotheses. *Make the semantics compute.* Modelling $\text{Sp}(2n, \mathbb{Z}_p)$ by transformations rather than matrices, with extensional equality of the action, made associativity and units definitional, let the ring solver discharge every generator obligation, and made the semidirect action definable from the same phase space the Pauli presentation targets, which is what allowed the two well-definedness conditions of the semidirect theorem to be proved by soundness and completeness rather than by hand.

7.4 On the use of a language model

A language model was used to help write the Agda code and to draft this paper from an outline written by the authors; the formalisation was designed, checked and corrected by the authors, and the paper was checked and revised by them. We record this for transparency, and because a verified development is one whose correctness does not depend on who, or what, wrote it.

8 RELATED WORK

Generators and relations for quantum gate groups. The programme of presenting quantum gate groups by generators and relations, with completeness proved through normal forms, was opened by Selinger’s presentation of the n -qubit Clifford group [Selinger 2015], whose staircase normal form and “push a generator through a box” rewrite system are the direct ancestors of the symplectic normal form verified here. The line continued with presentations of $U_n(\mathbb{Z}[\frac{1}{2}, i])$ by Bian and Selinger [Bian and Selinger 2021], building on Greylyn’s $U_4(\mathbb{Z}[1/\sqrt{2}, i])$ [Greylyn 2014]; of the orthogonal groups $O_n(\mathbb{Z}[\frac{1}{2}])$ by Li, Ross, and Selinger [Li et al. 2021]; of the real stabilizer operators by Makary, Ross, and Selinger [Makary et al. 2021]; of the 2-qubit Clifford+T and 3-qubit Clifford+CS groups by Bian and Selinger [Bian 2023; Bian and Selinger 2023a,b]; of the CNOT-dihedral operators by Amy, Chen, and Ross [Amy et al. 2018]; and of 3-qubit Toffoli–Hadamard circuits by Amy, Ross, and Wesley [Amy et al. 2024]. The first such result in an odd prime dimension is the qutrit Clifford rule set of Li, Mosca, Ross, van de Wetering, and Zhao [Li et al. 2025], and its generalisation to

all odd primes is the paper this work verifies [Bian et al. 2026a]. In every one of these papers the completeness proof is a normal-form argument checked by hand; the Bian–Selinger papers are the exception in that their *syntactic* relation-simplification steps were checked in Agda.

Complete equational theories for circuits. A parallel programme, phrased in terms of PROPs rather than groups, seeks complete equational theories for quantum circuits. Clément, Heurtel, Mansfield, Perdrix, and Valiron gave the first complete equational theory for arbitrary quantum circuits [Clément et al. 2023]; it was extended to ancillae and discarding [Clément et al. 2024b] and made minimal [Clément et al. 2024a]; and Clément’s complete theory for Real-Clifford+CH circuits comes with an appendix of 264 pages of equational derivations [Clément 2026]. Blake treats many near-Clifford fragments uniformly as PROPs, transferring completeness results and obtaining simpler and often minimal rule sets [Blake 2026c]; gives a finite, dimension-uniform presentation of qudit circuits via polycontrolled PROPs [Blake 2026a]; and proves completeness for prime-dimensional phase-affine circuits through unique normal forms [Blake 2026b]. All of these are pen-and-paper. Behind circuit presentations of this kind stands Lafont’s algebraic theory of Boolean circuits [Lafont 2003], whose normal forms for permutation circuits the circuit-level normal forms above generalise. On the graphical side, the ZX-calculus has complete axiomatisations for the qubit stabilizer fragment, for Clifford+T, and for the stabilizer fragment in odd prime dimensions [Backens 2014; Booth and Carette 2022; Jeandel et al. 2018; Poór et al. 2023]; as the paper we verify points out, completeness of a graphical calculus for linear maps does not yield a complete calculus for unitary circuits.

Mechanised completeness. Machine-checked completeness proofs for quantum circuit rule sets are rare. Bian and Selinger’s Agda developments [Bian and Selinger 2023a,b] verify that two *syntactic* presentations are equivalent and never mention a semantics. Choudhury, Karwowski, and Sabry mechanised, in HoTT-Agda, the presentation of the classical reversible language Π as a free symmetric rig groupoid [Choudhury et al. 2022], and Carette, Heunen, Kaarsgaard, and Sabry partially mechanised their square-root extension, including the completeness proof for the two-qubit Clifford fragment [Carette et al. 2024]; both are categorical coherence arguments rather than presentations of concrete groups. The library this paper builds on [Anonymous 2026b] was designed to close exactly this gap; the present paper is its first application to a research-level completeness theorem. Verified quantum compilers and program logics — QWIRE [Paykin et al. 2017], SQIR/VOQC [Hietala et al. 2021], VyZX [Lehmann et al. 2025], CoqQ [Zhou et al. 2023], Qbricks [Charetton et al. 2021] — prove the *soundness* of rewrites against a matrix semantics, never the completeness of a rule set; connecting our presented groups to such a semantics is the natural next step (§9).

Quantum programming and circuit tools. Closer to the programming-languages mainstream, a body of work takes circuits and gate-level rewriting as its object without asking for a completeness theorem. The verified compiler for quantum oracles of Li et al. [Li et al. 2022] proves in Rocq that oracle circuits are compiled semantics-preservingly; Proq [Li et al. 2020] adds projection-based runtime assertions to quantum programs; and QbC [Peduri et al. 2025] derives quantum while-programs from their specifications by refinement rules that are sound and complete for the refinement calculus, not for a gate set. Circuit optimisers and synthesisers such as Quarl [Li et al. 2024], Synthetiq [Paradis et al. 2024], and the component-based synthesiser of Kang and Oh [Kang and Oh 2023] search over gate-level rewrites and gate sets; a complete relation set is the guarantee that every pair of equivalent circuits is joined by a chain of such rewrites, so a verified one is a natural input for rewrite-based optimisers. Tower [Yuan and Carbin 2022], the quantum control machine [Yuan et al. 2024], and the network language PBKAT [Buckley et al. 2025] reason about

data structures, control flow, and protocols in superposition rather than about unitary groups. On the method side, our use of Agda as a library of reusable verified constructions, whose contribution lies in how the mechanisation is structured, follows the intrinsically-typed definitional interpreters à la carte of van der Rest et al. [van der Rest et al. 2022].

Group theory in proof assistants. Lean’s mathlib defines presented groups as quotients of free groups [The mathlib Community 2020, 2026], Isabelle’s Archive of Formal Proofs has free groups and the Nielsen–Schreier theorem [Breitner 2010; Kharim et al. 2023], and the Rocq/MathComp school [Gonthier et al. 2013] develops finite group theory over concrete permutation groups; none provides verified coset enumeration for a *given* finite relation set against a *given* concrete group. The classical algorithms are described by Sims [Sims 1994] and in the Handbook of Computational Group Theory [Holt et al. 2005], whose Proposition 2.55 is the pen-and-paper form of the extension theorems we use.

9 CONCLUSION AND FUTURE WORK

We have given a machine-checked proof that the qupit paper’s rule set presents the multi-qupit Clifford group in every odd prime dimension: modulo scalars, the paper’s Figure 1 presents $\mathcal{P}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$, and with the scalar ω and its single correction it presents a central extension of that group by $\langle \omega \rangle$; along the way, the simplified symplectic rules present $\text{Sp}(2n, \mathbb{Z}_p)$, and the symplectic normal form — refined into a doubly inductive datatype of boxes and rows — is unique. Every result is parametric in p , n and a primitive root. The proof does not follow the paper: it factors the group, presents the factors, and composes the presentations with generic verified theorems, and it connects back to the paper’s route by a verified isomorphism of presentations. We have argued that this organisation is not merely convenient for Agda but is the right way to structure such proofs: it confines the rewriting to the symplectic factor, where every rule is an honest equation, and it replaces the paper’s hand-performed Pauli bookkeeping by one theorem proved for arbitrary semidirect products.

Future work. The immediate target is the remaining glue: a verified matrix semantics for the qupit gates, against which the constructed groups of §7 would be shown isomorphic to the groups of unitaries, closing the chain from Figure 1 to $U_{p^n}(\mathbb{Z}[1/p, \omega])$ end to end. Beyond that, we intend to verify the other presentations of quantum gate groups in the literature (§8): the qubit Clifford group [Selinger 2015] is under way, with its own semidirect and extension presentations already in the tree, and the qutrit rule set [Li et al. 2025] is a special case of what is proved here at $p = 3$, up to a plumbing step for its own spelling of the rules. The presentation-level constructions used here and the supporting algebra are being prepared for submission to the Agda standard library.

REFERENCES

- Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A Finite Presentation of CNOT-Dihedral Operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97.
- Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A Sound and Complete Equational Theory for 3-Qubit Toffoli-Hadamard Circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. arXiv:2407.11152.
- Anonymous. 2026a. An Agda library for normal forms and complete relations for circuits. Supplementary material.
- Anonymous. 2026b. Normal Forms and Complete Relations for Circuits in Agda. Under submission; provided as supplementary material.
- Miriam Backens. 2014. The ZX-calculus is complete for stabilizer quantum mechanics. *New Journal of Physics* 16 (2014), 093021. arXiv:1307.7025.
- Xiaoning Bian. 2023. *Generators and Relations for Some Classes of Quantum Circuits*. Ph. D. Dissertation. Dalhousie University.

- Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026a. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions. Manuscript, July 2026. Agda code at <https://github.com/onestruggler/qupit>.
- Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026b. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions: Derived Relations and Supplemental Proofs. Supplement to [Bian et al. 2026a], July 2026.
- Xiaoning Bian and Peter Selinger. 2021. Generators and Relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. arXiv:2105.14047.
- Xiaoning Bian and Peter Selinger. 2023a. Generators and Relations for 2-Qubit Clifford+T Operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. arXiv:2204.02217.
- Xiaoning Bian and Peter Selinger. 2023b. Generators and Relations for 3-Qubit Clifford+CS Operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. arXiv:2306.08530.
- Colin Blake. 2026a. A Complete Equational Presentation of Qudit Circuits via Polycontrolled PROPs. In *51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (LIPIcs)*. arXiv:2602.09873.
- Colin Blake. 2026b. Completeness for Prime-Dimensional Phase-Affine Circuits. arXiv:2603.06466.
- Colin Blake. 2026c. Simpler Presentations for Many Fragments of Quantum Circuits. In *11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (LIPIcs)*. To appear. arXiv:2602.09874.
- Robert I. Booth and Titouan Carette. 2022. Complete ZX-Calculi for the Stabiliser Fragment in Odd Prime Dimensions. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022) (LIPIcs, Vol. 241)*. 24:1–24:15. arXiv:2204.12531.
- Joachim Breitner. 2010. Free Groups. Archive of Formal Proofs, <https://www.isa-afp.org/entries/Free-Groups.html>.
- Anita Buckley, Pavel Chuprikov, Rodrigo Otoni, Robert Soulé, Robert Rand, and Patrick Eugster. 2025. A Language for Quantifying Quantum Network Behavior. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025). <https://doi.org/10.1145/3763135>
- Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a Few Square Roots, Quantum Computing Is as Easy as Π . *Proceedings of the ACM on Programming Languages* 8, POPL (2024). arXiv:2310.14056.
- Christophe Charetton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An Automated Deductive Verification Framework for Circuit-Building Quantum Programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177.
- Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in Reversible Programming: From Symmetric Rig Groupoids to Reversible Programming Languages. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 6:1–6:32.
- Alexandre Clément. 2026. A Complete Equational Theory for Real-Clifford+CH Quantum Circuits. arXiv:2602.06644. Main text 13 pages, appendix of 264 pages of equational derivations.
- Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024a. Minimal Equational Theories for Quantum Circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. arXiv:2311.07476.
- Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024b. Quantum Circuit Completeness: Extensions and Simplifications. In *32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (LIPIcs, Vol. 288)*. 20:1–20:23. <https://doi.org/10.4230/LIPIcs.CSL.2024.20>
- Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A Complete Equational Theory for Quantum Circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. arXiv:2206.10577.
- Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. 2013. A Machine-Checked Proof of the Odd Order Theorem. In *Interactive Theorem Proving (ITP 2013) (Lecture Notes in Computer Science, Vol. 7998)*. 163–179.
- Seth E. M. Greylyn. 2014. Generators and relations for the group $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis, Dalhousie University. arXiv:1408.6204.
- Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A Verified Optimizer for Quantum Circuits. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 37:1–37:29.
- Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. 2005. *Handbook of Computational Group Theory*. Chapman & Hall/CRC.
- Emmanuel Jeandel, Simon Perdrix, and Renaud Vilmart. 2018. A Complete Axiomatisation of the ZX-Calculus for Clifford+T Quantum Mechanics. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*.

- Chan Gu Kang and Hakjoo Oh. 2023. Modular Component-Based Quantum Circuit Synthesis. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023). <https://doi.org/10.1145/3586039>
- Aabid Seeyal Abdul Kharim, Turga Venkata Hanumantha Prathamesh, Shweta Rajiv, and Rishi Vyas. 2023. Formalizing Free Groups in Isabelle/HOL: The Nielsen-Schreier Theorem and the Conjugacy Problem. In *Intelligent Computer Mathematics (CICM 2023) (Lecture Notes in Computer Science, Vol. 14101)*.
- Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310.
- Adrian Lehmann, Ben Caldwell, Bhakti Shah, and Robert Rand. 2025. VyZX: Formal Verification of a Graphical Quantum Language. *ACM Transactions on Programming Languages and Systems* (2025). arXiv:2311.11571.
- Gushu Li, Li Zhou, Nengkun Yu, Yufei Ding, Mingsheng Ying, and Yuan Xie. 2020. Projection-Based Runtime Assertions for Testing and Debugging Quantum Programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 150:1–150:29. <https://doi.org/10.1145/3428218>
- Liyi Li, Finn Voichick, Kesha Hietala, Yuxiang Peng, Xiaodi Wu, and Michael Hicks. 2022. Verified Compilation of Quantum Oracles. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 146:1–146:27. <https://doi.org/10.1145/3563309>
- Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A Complete and Natural Rule Set for Multi-Qutrit Clifford Circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670.
- Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and Relations for the Group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 210–264. arXiv:2106.01175.
- Zikun Li, Jinjun Peng, Yixuan Mei, Sina Lin, Yi Wu, Oded Padon, and Zhihao Jia. 2024. Quar: A Learning-Based Quantum Circuit Optimizer. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 114:1–114:29. <https://doi.org/10.1145/3649831>
- Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and Relations for Real Stabilizer Operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. arXiv:2109.05655.
- Ulf Norell. 2007. *Towards a Practical Programming Language Based on Dependent Type Theory*. Ph. D. Dissertation. Chalmers University of Technology.
- Anouk Paradis, Jasper Dekoninck, Benjamin Bichsel, and Martin Vechev. 2024. Synthetiq: Fast and Versatile Quantum Circuit Synthesis. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024). <https://doi.org/10.1145/3649813>
- Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858.
- Anurudh Peduri, Ina Schaefer, and Michael Walter. 2025. QbC: Quantum Correctness by Construction. *Proceedings of the ACM on Programming Languages* 9, OOPSLA1 (2025). <https://doi.org/10.1145/3720433>
- Boldizsár Poór, Robert I. Booth, Titouan Carrette, John van de Wetering, and Lia Yeh. 2023. The Qupit Stabiliser ZX-travaganza: Simplified Axioms, Normal Forms and Graph-Theoretic Simplification. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023)*. arXiv:2306.05204.
- Peter Selinger. 2015. Generators and relations for n -qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015). arXiv:1310.6813.
- Charles C. Sims. 1994. *Computation with Finitely Presented Groups*. Encyclopedia of Mathematics and its Applications, Vol. 48. Cambridge University Press.
- The Agda Community. 2026. The Agda standard library, version 2.3. <https://github.com/agda/agda-stdlib>.
- The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381.
- The mathlib Community. 2026. Mathlib documentation: PresentedGroup, CoxeterGroup, and the Nielsen-Schreier theorem. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/FreeGroup/NielsenSchreier.html.
- Cas van der Rest, Casper Bach Poulsen, Arjen Rouvoet, Eelco Visser, and Peter Mosses. 2022. Intrinsically-Typed Definitional Interpreters à la Carte. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 1903–1932. <https://doi.org/10.1145/3563355>
- Charles Yuan and Michael Carbin. 2022. Tower: Data Structures in Quantum Superposition. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022). <https://doi.org/10.1145/3563297>
- Charles Yuan, Agnes Villanyi, and Michael Carbin. 2024. Quantum Control Machine: The Limits of Control Flow in Quantum Programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024). <https://doi.org/10.1145/3649811>
- Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational Verification of Quantum Programs. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 29:1–29:33.